Complete ML Project Source Code

Predictive Analytics & AI Model Deployment - Customer Churn

23 files, 1,586 lines. Every source, configuration and documentation file in the project, in execution order. Generated data, trained models and figures are not source code and are in the project zip.

Table of contents

#	File	Purpose	Lines
1	README.md	Project overview and quick start	48
2	requirements.txt	Python dependencies	10
3	Dockerfile	Container image	7
4	Procfile	PaaS start command	1
5	.gitignore	Git ignore rules	4
6	run_pipeline.py	Runs the whole pipeline	19
7	src/config.py	Paths and constants	23
8	src/features.py	Cleaning + feature engineering + sklearn pipeline	80
9	src/data_collection.py	Task 1 - dataset	96
10	src/preprocess.py	Task 2 - preprocessing/EDA	101
11	src/train_models.py	Task 3 - train models	74
12	src/deep_learning.py	Task 5 - neural network	59
13	src/compare_models.py	Task 4 - compare/select model	112
14	src/predictive_analytics.py	Task 5 - importance, lift, segments, ROI	90
15	src/predict.py	Inference helper	30
16	src/build_dashboard.py	Task 7 - dashboard	69
17	src/build_docs.py	Task 8 - architecture diagram	39
18	src/build_pdfs.py	Builds the 8 task PDFs	427
19	src/build_source_pdf.py	Builds this PDF	82
20	app/app.py	Task 6 - Flask REST API	92
21	app/templates/index.html	Web form UI	37
22	tests/test_api.py	API tests	55
23	docs/ARCHITECTURE.md	Architecture notes	31

Run order: `python run_pipeline.py` then `python -m app.app`.

README.md (48 lines)

```
# Predictive Analytics & AI Model Deployment - Customer Churn

End-to-end machine-learning pipeline: data collection -> preprocessing/feature engineering ->
multi-model training -> comparison -> deep learning & predictive analytics -> Flask deployment ->
dashboard -> documentation.

**Business problem:** predict which subscription customers will churn so retention offers can be targeted.

> **Data note:** the build environment had no internet, so `src/data_collection.py` generates a
> 30,000-customer *synthetic* dataset (with injected missing values, outliers, duplicates, messy text).
> To use real data (e.g. Kaggle "Telco Customer Churn"), place a CSV at `data/raw/customer_churn_raw.csv`
> with the same column names and skip the generator.

## Quick start
```bash
pip install -r requirements.txt
python run_pipeline.py          # runs Tasks 1-5, 7, 8 (about 2 minutes on a laptop)
python -m unittest tests.test_api -v
python -m app.app               # http://127.0.0.1:5000  (form UI, /dashboard, REST API)
```
Production: `gunicorn -b 0.0.0.0:8000 "app.app:create_app()"` or `docker build -t churn . && docker run -p
      8000:8000 churn`.

API
Method	Path	Purpose
GET	`/health`	liveness check
GET	`/model-info`	model name, threshold, test metrics, accepted fields
POST	`/predict`	one customer (JSON) -> probability, label, risk level
POST	`/predict_batch`	JSON list or CSV upload (`file`) -> predictions
GET	`/dashboard`	analytics dashboard

```bash
curl -X POST localhost:5000/predict -H 'Content-Type: application/json' \
 -d '{"tenure_months":3,"monthly_charges":95,"contract_type":"Monthly"}'
```

Structure
```
src/           config, features (pipeline), tasks 1-5, dashboard, docs builders
app/           Flask API + web form
tests/         API tests
data/          raw/ and processed/ (train/test)
models/        all trained pipelines, best_model.joblib, metadata.json
results/       JSON/CSV outputs of every task
reports/       figures/, dashboard.html, test_predictions.csv
docs/          architecture.png, ARCHITECTURE.md, task PDFs (pdfs/)
```
See `docs/ARCHITECTURE.md` for design details and limitations.
```

requirements.txt (10 lines)

```
numpy>=2.0
pandas>=2.2
scikit-learn>=1.5
scipy>=1.11
joblib>=1.3
matplotlib>=3.8
seaborn>=0.13
Flask>=3.0
gunicorn>=22.0
reportlab>=4.0
```

Dockerfile (7 lines)

```
FROM python:3.12-slim
WORKDIR /srv
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "-w", "2", "app.app:create_app()"]
```

Procfile (1 lines)

```
web: gunicorn "app.app:create_app()" --bind 0.0.0.0:$PORT
```

.gitignore (4 lines)

```
__pycache__/
*.pyc
.venv/
.ipynb_checkpoints/
```

run_pipeline.py (19 lines)

```
"""Run the complete pipeline end-to-end:  python run_pipeline.py"""
import importlib
import time

STEPS = [("Task 1  Data collection", "src.data_collection"),
         ("Task 2  Preprocessing & feature engineering", "src.preprocess"),
         ("Task 3  Train multiple models", "src.train_models"),
         ("Task 5a Deep learning (MLP)", "src.deep_learning"),
         ("Task 4  Compare models / select best", "src.compare_models"),
         ("Task 5b Predictive analytics", "src.predictive_analytics"),
         ("Task 7  Dashboard", "src.build_dashboard"),
         ("Task 8  Architecture diagram", "src.build_docs")]

if __name__ == "__main__":
    for title, mod in STEPS:
        t0 = time.time(); print(f"\n=== {title} ===")
        importlib.import_module(mod).main()
        print(f"[done in {time.time() - t0:.0f}s]")
    print("\nAll done. Start the API with:  python -m app.app   (or gunicorn)")
```

src/config.py (23 lines)

```python
"""Central configuration: paths, constants and shared settings."""
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DATA_RAW = ROOT / "data" / "raw"
DATA_PROC = ROOT / "data" / "processed"
MODELS = ROOT / "models"
RESULTS = ROOT / "results"
REPORTS = ROOT / "reports"
FIGS = REPORTS / "figures"
DOCS = ROOT / "docs"

for _p in (DATA_RAW, DATA_PROC, MODELS, RESULTS, FIGS, DOCS):
    _p.mkdir(parents=True, exist_ok=True)

TARGET = "churn"
ID_COL = "customer_id"
SEED = 42
RAW_FILE = DATA_RAW / "customer_churn_raw.csv"

# Risk segmentation used by the dashboard and the API
RISK_BINS = [0.0, 0.25, 0.55, 1.0001]
RISK_LABELS = ["Low", "Medium", "High"]
```

src/features.py (80 lines)

```python
"""Cleaning rules, feature engineering and the sklearn preprocessing pipeline.

The full pipeline (feature engineering -> imputation/scaling/encoding -> model)
is a single sklearn Pipeline, so exactly the same steps run in training,
cross-validation and inside the deployed API (no train/serve skew, no leakage).
"""
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

RAW_NUMERIC = ["age", "tenure_months", "monthly_charges", "total_charges", "num_products",
               "num_support_tickets", "late_payments", "avg_monthly_usage_gb",
               "satisfaction_score", "last_login_days", "discount_offer"]
RAW_CATEGORICAL = ["gender", "region", "contract_type", "internet_service", "payment_method"]
ENG_NUMERIC = ["charge_per_product", "billing_consistency", "ticket_rate", "late_rate",
               "engagement_index", "low_satisfaction", "monthly_fiber_flag"]
ENG_CATEGORICAL = ["tenure_group", "age_group"]
NUMERIC = RAW_NUMERIC + ENG_NUMERIC
CATEGORICAL = RAW_CATEGORICAL + ENG_CATEGORICAL

# Physically plausible ranges: anything outside is treated as a data error -> NaN
VALID_RANGES = {
    "age": (18, 100), "tenure_months": (0, 120), "monthly_charges": (0, 250),
    "total_charges": (0, 30000), "num_products": (1, 10), "num_support_tickets": (0, 50),
    "late_payments": (0, 50), "avg_monthly_usage_gb": (0, 5000), "satisfaction_score": (1, 10),
    "last_login_days": (0, 365), "discount_offer": (0, 1),
}


def add_features(df: pd.DataFrame) -> pd.DataFrame:
    """Standardise raw columns, null out invalid values and engineer new features."""
    df = df.copy()
    for c in RAW_CATEGORICAL:
        if c not in df:
            df[c] = np.nan
        s = df[c].astype(object)
        df[c] = s.where(s.isna(), s.astype(str).str.strip().str.title())
        df[c] = df[c].where(df[c].notna(), np.nan)
    for c in RAW_NUMERIC:
        if c not in df:
            df[c] = np.nan
        df[c] = pd.to_numeric(df[c], errors="coerce")
        lo, hi = VALID_RANGES[c]
        df.loc[(df[c] < lo) | (df[c] > hi), c] = np.nan

    df["charge_per_product"] = df["monthly_charges"] / df["num_products"].clip(lower=1)
    df["billing_consistency"] = df["total_charges"] / (df["monthly_charges"] * df["tenure_months"] + 1)
    df["ticket_rate"] = df["num_support_tickets"] / (df["tenure_months"] + 1) * 12
    df["late_rate"] = df["late_payments"] / (df["tenure_months"] + 1) * 12
    df["engagement_index"] = df["avg_monthly_usage_gb"] / (df["last_login_days"] + 1)
    df["low_satisfaction"] = (df["satisfaction_score"] <= 4).astype(float).where(
        df["satisfaction_score"].notna(), np.nan)
    df["monthly_fiber_flag"] = ((df["contract_type"] == "Monthly") &
                                (df["internet_service"] == "Fiber")).astype(float)
    tg = pd.cut(df["tenure_months"], [-1, 12, 24, 48, 1000], labels=["0-12", "13-24", "25-48", "49+"])
    df["tenure_group"] = tg.astype(object)
    ag = pd.cut(df["age"], [0, 30, 45, 60, 200], labels=["<=30", "31-45", "46-60", "60+"])
    df["age_group"] = ag.astype(object)
    for c in ENG_CATEGORICAL:
        df[c] = df[c].where(df[c].notna(), "Unknown")
    return df[NUMERIC + CATEGORICAL]


def build_preprocessor() -> ColumnTransformer:
    num = Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())])
    cat = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
    return ColumnTransformer([("num", num, NUMERIC), ("cat", cat, CATEGORICAL)])


def make_pipeline(estimator) -> Pipeline:
    """Full end-to-end pipeline: features -> preprocessing -> estimator."""
    return Pipeline([
        ("features", FunctionTransformer(add_features, validate=False)),
        ("prep", build_preprocessor()),
        ("model", estimator),
    ])
```

src/data_collection.py (96 lines)

```python
"""TASK 1 - Collect a large business dataset.

No internet access was available in the build environment, so this module builds a
30,000-customer *synthetic* subscription-business dataset (business + sales + finance
flavour: billing, contracts, payments, support). The generator injects realistic data
quality problems (missing values, outliers, duplicates, messy text) so that Task 2 has
real cleaning work to do. To use a real dataset (e.g. Kaggle Telco Churn) just replace
data/raw/customer_churn_raw.csv keeping the same column names.
"""
import json
import numpy as np
import pandas as pd
from .config import RAW_FILE, RESULTS, SEED, TARGET


def _sigmoid(z):
    return 1 / (1 + np.exp(-z))


def generate(n: int = 30000, seed: int = SEED) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    age = np.clip(rng.normal(41, 13, n), 18, 85).round().astype(int)
    gender = rng.choice(["Male", "Female", "Other"], n, p=[.48, .49, .03])
    region = rng.choice(["North", "South", "East", "West", "Central"], n, p=[.22, .20, .20, .23, .15])
    contract = rng.choice(["Monthly", "One-Year", "Two-Year"], n, p=[.55, .25, .20])
    tenure = np.clip(rng.gamma(2.2, 14, n), 1, 72).round().astype(int)
    internet = rng.choice(["DSL", "Fiber", "No Service"], n, p=[.35, .50, .15])
    payment = rng.choice(["Electronic Check", "Credit Card", "Bank Transfer", "Mailed Check"], n,
                         p=[.34, .26, .24, .16])
    num_products = rng.integers(1, 7, n)
    base = pd.Series(internet).map({"DSL": 40, "Fiber": 70, "No Service": 18}).to_numpy()
    monthly = np.clip(base + 4 * num_products + rng.normal(0, 8, n), 15, 150).round(2)
    total = (monthly * tenure * rng.normal(1.0, 0.03, n)).round(2)
    tickets = rng.poisson(0.5 + 0.4 * (internet == "Fiber") + 0.15 * num_products)
    late = rng.poisson(0.3 + 0.7 * (payment == "Electronic Check") + 0.2 * (contract == "Monthly"))
    satisfaction = np.clip(np.round(7.2 - 0.45 * tickets - 0.4 * late + rng.normal(0, 1.6, n)), 1, 10)
    usage = np.round(rng.lognormal(4.0, 0.7, n) * (1 + 0.15 * num_products), 1)
    last_login = np.clip(rng.exponential(12, n) + 8 * (satisfaction < 5), 0, 180).round().astype(int)
    discount = rng.binomial(1, 0.22, n)

    def logit(b0):
        return (b0 + 1.1 * (contract == "Monthly") - 0.9 * (contract == "Two-Year") - 0.035 * tenure
                + 0.012 * (monthly - 65) + 0.22 * tickets + 0.35 * late - 0.28 * (satisfaction - 6)
                + 0.018 * last_login - 0.12 * num_products - 0.5 * discount
                + 0.6 * ((internet == "Fiber") & (monthly > 85)) + 0.25 * (payment == "Electronic Check")
                - 0.008 * (age - 41) + rng.normal(0, 0.6, n))

    lo, hi = -6.0, 4.0                      # bisection: calibrate intercept to ~27% churn
    for _ in range(30):
        mid = (lo + hi) / 2
        rate = _sigmoid(logit(mid)).mean()
        lo, hi = (mid, hi) if rate < 0.27 else (lo, mid)
    p = _sigmoid(logit((lo + hi) / 2))
    churn = rng.binomial(1, p)

    df = pd.DataFrame({
        "customer_id": [f"CUST{100000 + i}" for i in range(n)], "age": age, "gender": gender,
        "region": region, "tenure_months": tenure, "contract_type": contract,
        "internet_service": internet, "payment_method": payment, "num_products": num_products,
        "monthly_charges": monthly, "total_charges": total, "num_support_tickets": tickets,
        "late_payments": late, "avg_monthly_usage_gb": usage, "satisfaction_score": satisfaction,
        "last_login_days": last_login, "discount_offer": discount, TARGET: churn,
    })

    # ---- inject realistic data-quality problems ----
    for col, frac in [("total_charges", .03), ("satisfaction_score", .04), ("age", .02),
                      ("avg_monthly_usage_gb", .02), ("region", .01)]:
        df.loc[rng.random(n) < frac, col] = np.nan
    out = rng.random(n) < 0.006
    df.loc[out, "monthly_charges"] = df.loc[out, "monthly_charges"] * rng.choice([5, 10], out.sum())
    df.loc[rng.random(n) < 0.003, "age"] = 250                                  # impossible ages
    m = rng.random(n) < 0.05
    df.loc[m, "gender"] = df.loc[m, "gender"].str.upper()                       # 'MALE'
    m = rng.random(n) < 0.05
    df.loc[m, "contract_type"] = " " + df.loc[m, "contract_type"].str.lower() + " "  # ' monthly '
    df = pd.concat([df, df.sample(180, random_state=seed)], ignore_index=True)   # duplicates
    return df.sample(frac=1, random_state=seed).reset_index(drop=True)


def main():
    df = generate()
    df.to_csv(RAW_FILE, index=False)
    summary = {"rows": int(len(df)), "columns": int(df.shape[1]),
               "churn_rate_raw": round(float(df[TARGET].mean()), 4),
               "missing_cells": int(df.isna().sum().sum()),
               "duplicate_rows": int(df.duplicated().sum()),
               "file": str(RAW_FILE.name), "size_mb": round(RAW_FILE.stat().st_size / 1e6, 2),
               "dtypes": {k: str(v) for k, v in df.dtypes.items()},
               "missing_by_column": {k: int(v) for k, v in df.isna().sum().items() if v > 0},
```

```
                "head": df.head(8).round(2).astype(str).values.tolist(), "head_cols": list(df.columns)}
    (RESULTS / "task1_collection.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps({k: v for k, v in summary.items() if k not in ("head", "dtypes")}, indent=2))


if __name__ == "__main__":
    main()
```

src/preprocess.py (101 lines)

```python
"""TASK 2 - Advanced data preprocessing and feature engineering.

Steps: data-quality audit -> de-duplication -> stratified train/test split ->
fit the preprocessing pipeline on TRAIN only -> EDA figures + summary.
"""
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from .config import DATA_PROC, FIGS, RAW_FILE, RESULTS, SEED, TARGET
from .features import (CATEGORICAL, NUMERIC, RAW_NUMERIC, VALID_RANGES, add_features,
                       build_preprocessor)

sns.set_theme(style="whitegrid")


def audit(df):
    invalid = {}
    for c, (lo, hi) in VALID_RANGES.items():
        n = int(((df[c] < lo) | (df[c] > hi)).sum())
        if n:
            invalid[c] = n
    return {"rows": len(df), "duplicates": int(df.duplicated().sum()),
            "missing": {k: int(v) for k, v in df.isna().sum().items() if v},
            "out_of_range": invalid,
            "dirty_gender_values": int((df["gender"].dropna() != df["gender"].dropna().str.title()).sum()),
            "dirty_contract_values": int((df["contract_type"].dropna() !=
                                          df["contract_type"].dropna().str.strip().str.title()).sum())}


def main():
    raw = pd.read_csv(RAW_FILE)
    before = audit(raw)
    df = raw.drop_duplicates().dropna(subset=[TARGET]).reset_index(drop=True)
    train, test = train_test_split(df, test_size=0.2, stratify=df[TARGET], random_state=SEED)
    train.to_csv(DATA_PROC / "train.csv", index=False)
    test.to_csv(DATA_PROC / "test.csv", index=False)

    fe_train = add_features(train)
    after = {"missing": {k: int(v) for k, v in fe_train.isna().sum().items() if v}}
    prep = build_preprocessor().fit(fe_train)  # fit on train only (no leakage)
    Xt = prep.transform(fe_train)
    names = list(prep.get_feature_names_out())
    final_missing = int(np.isnan(Xt).sum())

    # ---------- figures ----------
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    m1 = pd.Series(before["missing"]).sort_values()
    m1.plot.barh(ax=ax[0], color="#d1495b"); ax[0].set_title("Missing values in raw data")
    ax[0].set_xlabel("count")
    cleaned = pd.Series(after["missing"]).sort_values()
    cleaned.plot.barh(ax=ax[1], color="#edae49"); ax[1].set_title("Missing after range validation")
    ax[1].set_xlabel("count (imputed in pipeline -> 0 remaining)")
    plt.tight_layout(); plt.savefig(FIGS / "t2_missing_values.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
    sns.boxplot(x=raw["monthly_charges"], ax=ax[0], color="#d1495b"); ax[0].set_title("monthly_charges - raw
      (outliers)")
    sns.boxplot(x=fe_train["monthly_charges"], ax=ax[1], color="#00798c"); ax[1].set_title("monthly_charges -
      validated")
    plt.tight_layout(); plt.savefig(FIGS / "t2_outliers.png", dpi=140); plt.close()

    tmp = fe_train.copy(); tmp[TARGET] = train[TARGET].values
    corr = tmp[NUMERIC + [TARGET]].corr()
    plt.figure(figsize=(9, 7.5))
    sns.heatmap(corr, cmap="RdBu_r", center=0, annot=True, fmt=".1f", annot_kws={"size": 6}, cbar_kws={"shrink":
       .7})
    plt.title("Correlation heatmap (raw + engineered features)")
    plt.tight_layout(); plt.savefig(FIGS / "t2_correlation.png", dpi=140); plt.close()

    c = corr[TARGET].drop(TARGET).sort_values()
    plt.figure(figsize=(7, 6))
    colors = ["#00798c" if v < 0 else "#d1495b" for v in c]
    c.plot.barh(color=colors); plt.title("Correlation of each numeric feature with churn")
    eng = set(["charge_per_product", "billing_consistency", "ticket_rate", "late_rate",
               "engagement_index", "low_satisfaction", "monthly_fiber_flag"])
    plt.gca().set_yticklabels([("* " if i in eng else "") + i for i in c.index])
    plt.xlabel("Pearson r  (* = engineered)"); plt.tight_layout()
    plt.savefig(FIGS / "t2_target_corr.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))
    for a, col in zip(ax, ["contract_type", "tenure_group", "payment_method"]):
        tmp.groupby(col)[TARGET].mean().sort_values().plot.bar(ax=a, color="#30638e")
        a.set_title(f"Churn rate by {col}"); a.set_ylabel("churn rate"); a.tick_params(axis="x", rotation=25)
    plt.tight_layout(); plt.savefig(FIGS / "t2_churn_by_segment.png", dpi=140); plt.close()
```

```
    summary = {"raw_rows": before["rows"], "clean_rows": len(df), "train_rows": len(train),
               "test_rows": len(test), "audit_before": before, "missing_after_validation": after["missing"],
               "final_matrix_shape": list(Xt.shape), "final_missing": final_missing,
               "n_numeric_raw": len(RAW_NUMERIC), "n_numeric_total": len(NUMERIC),
               "n_categorical": len(CATEGORICAL), "encoded_feature_count": len(names),
               "churn_rate_train": round(float(train[TARGET].mean()), 4),
               "churn_rate_test": round(float(test[TARGET].mean()), 4),
               "top_corr": {k: round(float(v), 3) for k, v in
        c.abs().sort_values(ascending=False).head(8).index.to_series().map(lambda k: c[k]).items()}}
    (RESULTS / "task2_preprocess.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

src/train_models.py (74 lines)

```python
"""TASK 3 - Train multiple ML models with scikit-learn.

Every model is a full pipeline (feature engineering + preprocessing + estimator), evaluated
with stratified 3-fold cross-validation on the training set, then fitted and saved.
A HistGradientBoosting model is additionally tuned with RandomizedSearchCV.
"""
import json
import time
import joblib
import pandas as pd
from scipy.stats import loguniform
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from .config import DATA_PROC, MODELS, RESULTS, SEED, TARGET
from .features import make_pipeline

CV = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)


def candidate_models():
    return {
        "logistic_regression": LogisticRegression(max_iter=2000, C=1.0),
        "decision_tree": DecisionTreeClassifier(max_depth=7, min_samples_leaf=40, random_state=SEED),
        "random_forest": RandomForestClassifier(n_estimators=200, min_samples_leaf=5, max_features="sqrt",
                                                n_jobs=-1, random_state=SEED),
        "gradient_boosting": HistGradientBoostingClassifier(learning_rate=0.06, max_iter=250,
                                                            random_state=SEED),
        "knn": KNeighborsClassifier(n_neighbors=35, weights="distance", n_jobs=-1),
    }


def load_train():
    train = pd.read_csv(DATA_PROC / "train.csv")
    return train.drop(columns=[TARGET]), train[TARGET]


def main():
    X, y = load_train()
    results = {}
    for name, est in candidate_models().items():
        pipe = make_pipeline(est)
        t0 = time.time()
        scores = cross_val_score(pipe, X, y, cv=CV, scoring="roc_auc")
        pipe.fit(X, y)
        results[name] = {"cv_roc_auc_mean": round(float(scores.mean()), 4),
                         "cv_roc_auc_std": round(float(scores.std()), 4),
                         "train_seconds": round(time.time() - t0, 1)}
        joblib.dump(pipe, MODELS / f"{name}.joblib")
        print(name, results[name], flush=True)

    # hyper-parameter tuning
    t0 = time.time()
    search = RandomizedSearchCV(
        make_pipeline(HistGradientBoostingClassifier(random_state=SEED)),
        {"model__learning_rate": loguniform(0.02, 0.2), "model__max_iter": [150, 250, 400],
         "model__max_leaf_nodes": [8, 15, 31], "model__min_samples_leaf": [20, 50, 100],
         "model__l2_regularization": loguniform(1e-3, 10)},
        n_iter=10, cv=CV, scoring="roc_auc", random_state=SEED, n_jobs=1, refit=True)
    search.fit(X, y)
    joblib.dump(search.best_estimator_, MODELS / "gradient_boosting_tuned.joblib")
    results["gradient_boosting_tuned"] = {
        "cv_roc_auc_mean": round(float(search.best_score_), 4), "cv_roc_auc_std": None,
        "train_seconds": round(time.time() - t0, 1),
        "best_params": {k.replace("model__", ""): (round(v, 5) if isinstance(v, float) else v)
                        for k, v in search.best_params_.items()}}
    print("tuned", results["gradient_boosting_tuned"], flush=True)
    (RESULTS / "task3_training.json").write_text(json.dumps(results, indent=2))


if __name__ == "__main__":
    main()
```

src/deep_learning.py (59 lines)

```python
"""TASK 5a - Deep learning: a multilayer neural network (MLP) on the same pipeline.

TensorFlow/PyTorch are not installed in this environment, so the network is built with
scikit-learn's MLPClassifier (dense layers, ReLU, Adam, L2, early stopping). The code is
structured so the estimator can be swapped for a Keras model wrapped with SciKeras.
"""
import json
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from .config import FIGS, MODELS, RESULTS, SEED
from .features import make_pipeline
from .train_models import load_train

ARCHS = {"64": (64,), "128-64": (128, 64), "128-64-32": (128, 64, 32)}


def mlp(arch):
    return MLPClassifier(hidden_layer_sizes=arch, activation="relu", solver="adam", alpha=1e-3,
                         batch_size=256, learning_rate_init=1e-3, early_stopping=True,
                         validation_fraction=0.1, n_iter_no_change=10, max_iter=200, random_state=SEED)


def main():
    X, y = load_train()
    Xa, Xv, ya, yv = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
    val = {}
    for name, arch in ARCHS.items():
        p = make_pipeline(mlp(arch)).fit(Xa, ya)
        val[name] = round(float(roc_auc_score(yv, p.predict_proba(Xv)[:, 1])), 4)
        print("arch", name, val[name], flush=True)
    best = max(val, key=val.get)
    final = make_pipeline(mlp(ARCHS[best])).fit(X, y)       # refit on full train set
    net = final.named_steps["model"]
    joblib.dump(final, MODELS / "mlp_neural_net.joblib")

    fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
    ax[0].plot(net.loss_curve_, color="#d1495b"); ax[0].set_title("Training loss (log-loss)")
    ax[0].set_xlabel("epoch"); ax[0].set_ylabel("loss")
    ax[1].plot(net.validation_scores_, color="#00798c"); ax[1].set_title("Validation accuracy (early stopping)")
    ax[1].set_xlabel("epoch"); ax[1].set_ylabel("accuracy")
    plt.tight_layout(); plt.savefig(FIGS / "t5_mlp_training.png", dpi=140); plt.close()

    out = {"architectures_val_auc": val, "selected": best, "layers": list(ARCHS[best]),
           "activation": "relu", "optimizer": "adam", "l2_alpha": 1e-3, "batch_size": 256,
           "epochs_run": int(net.n_iter_), "best_val_accuracy": round(float(net.best_validation_score_), 4),
           "final_train_loss": round(float(net.loss_), 4),
           "n_parameters": int(sum(w.size for w in net.coefs_) + sum(b.size for b in net.intercepts_))}
    (RESULTS / "task5_deep_learning.json").write_text(json.dumps(out, indent=2))
    print(json.dumps(out, indent=2))


if __name__ == "__main__":
    main()
```

src/compare_models.py (112 lines)

```python
"""TASK 4 - Compare model accuracy and performance metrics on the held-out test set.

All saved pipelines are scored on data never used for fitting/tuning. The best model
(by ROC-AUC) is promoted to models/best_model.joblib; its decision threshold is chosen
on out-of-fold TRAIN predictions (maximising F1) to avoid tuning on the test set.
"""
import json
from datetime import datetime, timezone
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.calibration import calibration_curve
from sklearn.metrics import (accuracy_score, average_precision_score, brier_score_loss, confusion_matrix,
                             f1_score, log_loss, precision_recall_curve, precision_score, recall_score,
                             roc_auc_score, roc_curve)
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from .config import DATA_PROC, MODELS, RESULTS, FIGS, SEED, TARGET
from .features import CATEGORICAL, NUMERIC

PRETTY = {"logistic_regression": "Logistic Regression", "decision_tree": "Decision Tree",
          "random_forest": "Random Forest", "gradient_boosting": "Gradient Boosting",
          "gradient_boosting_tuned": "Gradient Boosting (tuned)", "knn": "k-NN",
          "mlp_neural_net": "Neural Network (MLP)"}


def metrics(y, p, thr=0.5):
    pred = (p >= thr).astype(int)
    return {"accuracy": accuracy_score(y, pred), "precision": precision_score(y, pred),
            "recall": recall_score(y, pred), "f1": f1_score(y, pred), "roc_auc": roc_auc_score(y, p),
            "pr_auc": average_precision_score(y, p), "log_loss": log_loss(y, p),
            "brier": brier_score_loss(y, p)}


def main():
    test = pd.read_csv(DATA_PROC / "test.csv"); train = pd.read_csv(DATA_PROC / "train.csv")
    Xte, yte = test.drop(columns=[TARGET]), test[TARGET].values
    probs, rows = {}, []
    for f in sorted(MODELS.glob("*.joblib")):
        if f.stem == "best_model":
            continue
        pipe = joblib.load(f)
        p = pipe.predict_proba(Xte)[:, 1]
        probs[f.stem] = p
        rows.append({"model": f.stem, "name": PRETTY.get(f.stem, f.stem), **metrics(yte, p)})
    df = pd.DataFrame(rows).sort_values("roc_auc", ascending=False).reset_index(drop=True)
    df.round(4).to_csv(RESULTS / "model_comparison.csv", index=False)
    best = df.loc[0, "model"]

    best_pipe = joblib.load(MODELS / f"{best}.joblib")
    oof = cross_val_predict(best_pipe, train.drop(columns=[TARGET]), train[TARGET],
                            cv=StratifiedKFold(3, shuffle=True, random_state=SEED), method="predict_proba")[:,
      1]
    grid = np.linspace(0.1, 0.7, 61)
    thr = float(grid[int(np.argmax([f1_score(train[TARGET], oof >= t) for t in grid]))])
    bm = metrics(yte, probs[best], thr)
    joblib.dump(best_pipe, MODELS / "best_model.joblib")
    meta = {"best_model": best, "best_model_name": PRETTY.get(best, best), "threshold": round(thr, 3),
            "test_metrics_at_threshold": {k: round(float(v), 4) for k, v in bm.items()},
            "trained_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
            "numeric_features": NUMERIC, "categorical_features": CATEGORICAL, "target": TARGET}
    (MODELS / "metadata.json").write_text(json.dumps(meta, indent=2))
    cm = confusion_matrix(yte, probs[best] >= thr)

    # ---------- figures ----------
    pal = sns.color_palette("tab10", len(df))
    plt.figure(figsize=(6.5, 5.5))
    for c, (m, p) in zip(pal, probs.items()):
        fpr, tpr, _ = roc_curve(yte, p)
        plt.plot(fpr, tpr, color=c, label=f"{PRETTY.get(m, m)} ({roc_auc_score(yte, p):.3f})")
    plt.plot([0, 1], [0, 1], "k--", lw=.8); plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
    plt.title("ROC curves (test set)"); plt.legend(fontsize=7, loc="lower right"); plt.tight_layout()
    plt.savefig(FIGS / "t4_roc.png", dpi=140); plt.close()

    plt.figure(figsize=(6.5, 5.5))
    for c, (m, p) in zip(pal, probs.items()):
        pr, rc, _ = precision_recall_curve(yte, p)
        plt.plot(rc, pr, color=c, label=f"{PRETTY.get(m, m)} ({average_precision_score(yte, p):.3f})")
    plt.axhline(yte.mean(), color="k", ls="--", lw=.8); plt.xlabel("Recall"); plt.ylabel("Precision")
    plt.title("Precision-Recall curves (test set)"); plt.legend(fontsize=7); plt.tight_layout()
    plt.savefig(FIGS / "t4_pr.png", dpi=140); plt.close()

    long = df.melt(id_vars="name", value_vars=["accuracy", "precision", "recall", "f1", "roc_auc"])
    plt.figure(figsize=(10, 4.6))
    sns.barplot(data=long, x="name", y="value", hue="variable")
    plt.xticks(rotation=25, ha="right"); plt.ylim(0.3, 0.9); plt.xlabel(""); plt.ylabel("score")
```

```
    plt.title("Model comparison - test set (threshold 0.5)"); plt.legend(ncol=5, fontsize=8)
    plt.tight_layout(); plt.savefig(FIGS / "t4_metric_bars.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 2, figsize=(11, 4.4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax[0], xticklabels=["Stay", "Churn"],
                yticklabels=["Stay", "Churn"]); ax[0].set_title(f"Confusion matrix - {PRETTY.get(best, best)}\n(threshold {thr:.2f})")
    ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual")
    for c, (m, p) in zip(pal, probs.items()):
        fp, mp = calibration_curve(yte, p, n_bins=10, strategy="quantile")
        ax[1].plot(mp, fp, marker="o", ms=3, color=c, label=PRETTY.get(m, m))
    ax[1].plot([0, 1], [0, 1], "k--", lw=.8); ax[1].set_title("Calibration (reliability) curves")
    ax[1].set_xlabel("Mean predicted probability"); ax[1].set_ylabel("Observed churn rate"); ax[1].legend(fontsize=6)
    plt.tight_layout(); plt.savefig(FIGS / "t4_confusion_calibration.png", dpi=140); plt.close()

    out = {"table": df.round(4).to_dict("records"), "best_model": best, "threshold": thr,
           "best_at_threshold": meta["test_metrics_at_threshold"], "confusion_matrix": cm.tolist(),
           "test_rows": int(len(yte)), "test_churn_rate": round(float(yte.mean()), 4),
           "majority_baseline_accuracy": round(float(1 - yte.mean()), 4)}
    (RESULTS / "task4_comparison.json").write_text(json.dumps(out, indent=2))
    print(df.round(4).drop(columns="model").to_string()); print("BEST:", best, "thr", thr, meta["test_metrics_at_threshold"])


if __name__ == "__main__":
    main()
```

src/predictive_analytics.py (90 lines)

```
"""TASK 5b - Predictive analytics on the selected model.

Permutation feature importance, decile lift / cumulative gains, risk segmentation and an
illustrative retention-campaign ROI simulation. Writes reports/test_predictions.csv that
feeds the dashboard.
"""
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.inspection import permutation_importance
from .config import DATA_PROC, FIGS, MODELS, REPORTS, RESULTS, RISK_BINS, RISK_LABELS, SEED, TARGET

COST_PER_OFFER, VALUE_OF_SAVED_CUSTOMER, SAVE_RATE = 20.0, 400.0, 0.30   # illustrative assumptions


def main():
    meta = json.loads((MODELS / "metadata.json").read_text())
    pipe = joblib.load(MODELS / "best_model.joblib")
    test = pd.read_csv(DATA_PROC / "test.csv")
    X, y = test.drop(columns=[TARGET]), test[TARGET]
    p = pipe.predict_proba(X)[:, 1]

    pred = pd.DataFrame({"customer_id": test["customer_id"], "churn_probability": p.round(4),
                         "predicted_churn": (p >= meta["threshold"]).astype(int), "actual_churn": y})
    pred["risk_segment"] = pd.cut(pred["churn_probability"], RISK_BINS, labels=RISK_LABELS, right=False)
    pred.to_csv(REPORTS / "test_predictions.csv", index=False)

    # 1) permutation importance on the RAW input columns
    feats = [c for c in X.columns if c != "customer_id"]
    idx = X.sample(3000, random_state=SEED).index
    imp = permutation_importance(pipe, X.loc[idx, feats], y.loc[idx], scoring="roc_auc",
                                 n_repeats=5, random_state=SEED, n_jobs=1)
    imp_df = pd.DataFrame({"feature": feats, "importance": imp.importances_mean,
                           "std": imp.importances_std}).sort_values("importance", ascending=False)
    top = imp_df.head(12).iloc[::-1]
    plt.figure(figsize=(7, 5)); plt.barh(top["feature"], top["importance"], xerr=top["std"], color="#30638e")
    plt.xlabel("Drop in ROC-AUC when feature is shuffled"); plt.title("Permutation feature importance (top 12)")
    plt.tight_layout(); plt.savefig(FIGS / "t5_importance.png", dpi=140); plt.close()

    # 2) decile lift / gains
    pred["decile"] = pd.qcut(pred["churn_probability"].rank(method="first", ascending=False), 10,
      labels=range(1, 11))
    base = pred["actual_churn"].mean()
    dec = pred.groupby("decile", observed=True).agg(customers=("actual_churn", "size"),
      churners=("actual_churn", "sum"),
                                                    avg_prob=("churn_probability", "mean")).reset_index()
    dec["churn_rate"] = dec["churners"] / dec["customers"]
    dec["lift"] = dec["churn_rate"] / base
    dec["cum_gain"] = dec["churners"].cumsum() / dec["churners"].sum()
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    ax[0].bar(dec["decile"].astype(int), dec["lift"], color="#00798c"); ax[0].axhline(1, color="k", ls="--",
      lw=.8)
    ax[0].set_title("Lift by risk decile (1 = highest risk)"); ax[0].set_xlabel("decile");
      ax[0].set_ylabel("lift vs. average")
    ax[1].plot([0] + list(range(10, 101, 10)), [0] + list(dec["cum_gain"] * 100), marker="o", color="#d1495b",
      label="model")
    ax[1].plot([0, 100], [0, 100], "k--", lw=.8, label="random"); ax[1].legend()
    ax[1].set_title("Cumulative gains"); ax[1].set_xlabel("% customers contacted"); ax[1].set_ylabel("% churners
       captured")
    plt.tight_layout(); plt.savefig(FIGS / "t5_lift_gains.png", dpi=140); plt.close()

    # 3) risk segments
    seg = pred.groupby("risk_segment", observed=True).agg(customers=("actual_churn", "size"),
      avg_pred=("churn_probability", "mean"),
                                                          actual_rate=("actual_churn", "mean")).reset_index()
    seg["share"] = seg["customers"] / len(pred)
    plt.figure(figsize=(6.5, 4)); x = np.arange(len(seg)); w = .38
    plt.bar(x - w / 2, seg["avg_pred"], w, label="Predicted", color="#edae49")
    plt.bar(x + w / 2, seg["actual_rate"], w, label="Actual", color="#30638e")
    plt.xticks(x, seg["risk_segment"].astype(str)); plt.ylabel("churn rate"); plt.title("Risk segments:
      predicted vs. actual churn")
    plt.legend(); plt.tight_layout(); plt.savefig(FIGS / "t5_segments.png", dpi=140); plt.close()

    # 4) illustrative ROI of contacting the top-k% by risk
    order = pred.sort_values("churn_probability", ascending=False).reset_index(drop=True)
    roi = []
    for k in [5, 10, 20, 30, 40, 50]:
        top_k = order.head(int(len(order) * k / 100))
        saved = top_k["actual_churn"].sum() * SAVE_RATE
        net = saved * VALUE_OF_SAVED_CUSTOMER - len(top_k) * COST_PER_OFFER
        roi.append({"top_pct": k, "contacted": len(top_k), "churners_reached": int(top_k["actual_churn"].sum()),
                    "expected_saved": round(saved, 1), "net_value": round(net, 0)})
    out = {"importance": imp_df.head(12).round(4).to_dict("records"), "deciles":
      dec.round(4).to_dict("records"),
           "segments": seg.assign(risk_segment=seg["risk_segment"].astype(str)).round(4).to_dict("records"),
```

```
            "roi": roi, "assumptions": {"cost_per_offer": COST_PER_OFFER, "value_of_saved_customer": VALUE_OF_SAVED_CUSTOMER,
                                        "save_rate": SAVE_RATE}, "top_decile_lift": round(float(dec.loc[0, "lift"]), 2),
            "top_20pct_gain": round(float(dec.loc[1, "cum_gain"]), 3)}
    (RESULTS / "task5_analytics.json").write_text(json.dumps(out, indent=2))
    print(json.dumps({k: out[k] for k in ["segments", "roi", "top_decile_lift", "top_20pct_gain"]}, indent=1))
    print(imp_df.head(8).round(4).to_string())


if __name__ == "__main__":
    main()
```

src/predict.py (30 lines)

```
"""Inference helpers shared by the Flask API, tests and notebooks."""
import json
from functools import lru_cache
import joblib
import pandas as pd
from .config import MODELS, RISK_BINS, RISK_LABELS


@lru_cache(maxsize=1)
def load_artifacts():
    model = joblib.load(MODELS / "best_model.joblib")
    meta = json.loads((MODELS / "metadata.json").read_text())
    return model, meta


def risk_level(p: float) -> str:
    for lo, hi, lab in zip(RISK_BINS[:-1], RISK_BINS[1:], RISK_LABELS):
        if lo <= p < hi:
            return lab
    return RISK_LABELS[-1]


def predict_records(records):
    """records: list[dict] with raw customer fields -> list of prediction dicts."""
    model, meta = load_artifacts()
    df = pd.DataFrame(records)
    probs = model.predict_proba(df)[:, 1]
    return [{"churn_probability": round(float(p), 4), "prediction": int(p >= meta["threshold"]),
             "label": "Will churn" if p >= meta["threshold"] else "Will stay",
             "risk_level": risk_level(float(p)), "threshold": meta["threshold"]} for p in probs]
```

src/build_dashboard.py (69 lines)

```python
"""TASK 7 - Build a self-contained HTML dashboard + prediction-analysis report."""
import base64
import json
import pandas as pd
from .config import FIGS, MODELS, REPORTS, RESULTS


def img(name, alt=""):
    b = base64.b64encode((FIGS / name).read_bytes()).decode()
    return f'<img alt="{alt}" src="{b}">'


def table(df, cls=""):
    return df.to_html(index=False, border=0, classes=cls, float_format=lambda v: f"{v:.3f}")


def main():
    meta = json.loads((MODELS / "metadata.json").read_text())
    t1 = json.loads((RESULTS / "task1_collection.json").read_text())
    t2 = json.loads((RESULTS / "task2_preprocess.json").read_text())
    t4 = json.loads((RESULTS / "task4_comparison.json").read_text())
    t5 = json.loads((RESULTS / "task5_analytics.json").read_text())
    pred = pd.read_csv(REPORTS / "test_predictions.csv")
    m = meta["test_metrics_at_threshold"]
    high = int((pred["risk_segment"] == "High").sum())

    cmp = pd.DataFrame(t4["table"])[["name", "accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc"]]
    cmp.columns = ["Model", "Accuracy", "Precision", "Recall", "F1", "ROC-AUC", "PR-AUC"]
    seg = pd.DataFrame(t5["segments"])[["risk_segment", "customers", "share", "avg_pred", "actual_rate"]]
    seg.columns = ["Segment", "Customers", "Share", "Avg predicted", "Actual churn"]
    roi = pd.DataFrame(t5["roi"]); roi.columns = ["Top % contacted", "Contacted", "Churners reached", "Expected saved", "Net value ($)"]
    top = pred.sort_values("churn_probability", ascending=False).head(15)[["customer_id", "churn_probability", "risk_segment", "actual_churn"]]
    top.columns = ["Customer", "Churn probability", "Segment", "Actually churned"]

    kpis = [("Customers analysed", f"{t2['clean_rows']:,}"), ("Overall churn rate", f"{t2['churn_rate_train']*100:.1f}%"),
            ("Best model", meta["best_model_name"]), ("ROC-AUC (test)", f"{m['roc_auc']:.3f}"),
            ("Recall @ thr %.2f" % meta["threshold"], f"{m['recall']*100:.0f}%"), ("High-risk customers (test)", f"{high:,}")]
    cards = "".join(f'<div class="kpi"><div class="v">{v}</div><div class="l">{k}</div></div>' for k, v in kpis)
    html = f"""<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Churn Prediction Dashboard</title><style>
body{{font-family:system-ui,Segoe UI,Arial,sans-serif;background:#f4f6f8;margin:0;color:#1d2733}}
header{{background:#1f3a5f;color:#fff;padding:20px 28px}}h1{{margin:0;font-size:22px}}header p{{margin:4px 0 0;opacity:.8;font-size:13px}}
main{{max-width:1150px;margin:18px auto;padding:0 16px}}h2{{font-size:17px;margin:26px 0 8px;color:#1f3a5f}}
.kpis{{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px}}
.kpi{{background:#fff;border-radius:10px;padding:16px;box-shadow:0 1px 4px rgba(0,0,0,.08)}}.kpi .v{{font-size:22px;font-weight:700;color:#1f3a5f}}.kpi .l{{font-size:12px;color:#5c6b7a;margin-top:4px}}
.grid{{display:grid;grid-template-columns:repeat(auto-fit,minmax(480px,1fr));gap:14px}}
.card{{background:#fff;border-radius:10px;padding:14px;box-shadow:0 1px 4px rgba(0,0,0,.08);overflow-x:auto}}.card img{{width:100%;height:auto}}
table{{border-collapse:collapse;width:100%;font-size:13px}}th{{background:#e9eef4;text-align:left}}th,td{{padding:6px 9px;border-bottom:1px solid #e3e8ee}}
.note{{font-size:12px;color:#5c6b7a;margin-top:8px}}
</style></head><body><header><h1>Predictive Analytics &amp; AI Model Deployment - Churn Dashboard</h1>
<p>Best model: {meta['best_model_name']} &middot; threshold {meta['threshold']} &middot; generated {meta['trained_at'][:10]} &middot; data: synthetic 30,000-customer subscription dataset</p></header><main>
<div class="kpis">{cards}</div>
<h2>1. Data &amp; drivers</h2><div class="grid"><div class="card">{img('t2_churn_by_segment.png')}</div><div class="card">{img('t2_target_corr.png')}</div></div>
<h2>2. Model comparison</h2><div class="card">{table(cmp)}<p class="note">Held-out test set of {t4['test_rows']:,} customers; metrics at threshold 0.5. Majority-class accuracy baseline = {t4['majority_baseline_accuracy']:.3f}.</p></div>
<div class="grid" style="margin-top:14px"><div class="card">{img('t4_metric_bars.png')}</div><div class="card">{img('t4_roc.png')}</div></div>
<h2>3. Prediction analysis</h2><div class="grid"><div class="card">{img('t4_confusion_calibration.png')}</div><div class="card">{img('t5_importance.png')}</div>
<div class="card">{img('t5_lift_gains.png')}</div><div class="card">{img('t5_segments.png')}</div></div>
<h2>4. Risk segments</h2><div class="card">{table(seg)}</div>
<h2>5. Retention campaign simulation (illustrative)</h2><div class="card">{table(roi)}
<p class="note">Assumptions: ${t5['assumptions']['cost_per_offer']:.0f} per offer, ${t5['assumptions']['value_of_saved_customer']:.0f} value per saved customer, {t5['assumptions']['save_rate']*100:.0f}% of contacted churners retained. These are placeholders - replace with real business numbers.</p></div>
<h2>6. Highest-risk customers in the test set</h2><div class="card">{table(top)}</div>
<h2>7. Neural network training</h2><div class="card">{img('t5_mlp_training.png')}</div>
</main></body></html>"""
    (REPORTS / "dashboard.html").write_text(html, encoding="utf-8")
    print("dashboard.html", round(len(html) / 1e6, 2), "MB")


if __name__ == "__main__":
    main()
```

src/build_docs.py (39 lines)

```
"""TASK 8 - Generate the architecture diagram used in the documentation."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
from .config import DOCS

BOXES = [  # (x, y, w, h, title, subtitle, color)
    (0.2, 3.2, 2.3, 1.3, "1. Data Collection", "src/data_collection.py\n30k customer records", "#d6e6f5"),
    (3.0, 3.2, 2.3, 1.3, "2. Preprocessing", "src/preprocess.py + features.py\nclean - split - engineer",
      "#d6e6f5"),
    (5.8, 3.2, 2.3, 1.3, "3. Model Training", "src/train_models.py\n5 models + tuned GBM", "#d9ead3"),
    (8.6, 3.2, 2.3, 1.3, "5a. Deep Learning", "src/deep_learning.py\nMLP 128-64", "#d9ead3"),
    (8.6, 1.4, 2.3, 1.3, "4. Model Comparison", "src/compare_models.py\nmetrics - best - threshold", "#fff2cc"),
    (5.8, 1.4, 2.3, 1.3, "5b. Predictive Analytics", "permutation importance\nlift, segments, ROI", "#fff2cc"),
    (3.0, 1.4, 2.3, 1.3, "7. Dashboard", "src/build_dashboard.py\nreports/dashboard.html", "#f4cccc"),
    (0.2, 1.4, 2.3, 1.3, "6. Deployment", "app/app.py (Flask)\n/predict /predict_batch", "#f4cccc"),
]


def main():
    fig, ax = plt.subplots(figsize=(12, 5.6)); ax.set_xlim(0, 11.2); ax.set_ylim(0, 5.4); ax.axis("off")
    for x, y, w, h, t, s, c in BOXES:
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.03,rounding_size=0.12", fc=c,
      ec="#33404d", lw=1.2))
        ax.text(x + w / 2, y + h * .68, t, ha="center", va="center", fontsize=10, fontweight="bold")
        ax.text(x + w / 2, y + h * .3, s, ha="center", va="center", fontsize=7.2, color="#33404d")
    arrows = [(2.5, 3.85, 3.0, 3.85), (5.3, 3.85, 5.8, 3.85), (8.1, 3.85, 8.6, 3.85), (9.75, 3.2, 9.75, 2.7),
              (8.6, 2.05, 8.1, 2.05), (5.8, 2.05, 5.3, 2.05), (3.0, 2.05, 2.5, 2.05)]
    for a in arrows:
        ax.annotate("", xy=a[2:], xytext=a[:2], arrowprops=dict(arrowstyle="-|>", lw=1.6, color="#33404d"))
    ax.text(5.6, 5.05, "AI Pipeline Architecture - Customer Churn Prediction", ha="center", fontsize=13,
      fontweight="bold")
    ax.text(5.6, 0.85, "Artifacts:  data/raw -> data/processed -> models/*.joblib -> best_model.joblib +
      metadata.json"
            " -> reports/ -> REST API / web UI", ha="center", fontsize=8.2, color="#33404d")
    ax.text(5.6, 0.45, "Each model is ONE sklearn Pipeline: add_features -> impute/scale/encode -> estimator
      (same in training and serving)",
            ha="center", fontsize=8.2, color="#33404d")
    plt.tight_layout(); plt.savefig(DOCS / "architecture.png", dpi=150); plt.close()


if __name__ == "__main__":
    main()
```

src/build_pdfs.py (427 lines)

```
"""Builds one PDF report per project task (docs/pdfs/Task_N_*.pdf) from the saved results."""
import json
import textwrap
from pathlib import Path
from PIL import Image as PILImage
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.platypus import (Image, KeepTogether, PageBreak, Paragraph, Preformatted, SimpleDocTemplate,
                                Spacer, Table, TableStyle)
from .config import DOCS, FIGS, MODELS, RESULTS, ROOT

OUT = DOCS / "pdfs"; OUT.mkdir(parents=True, exist_ok=True)
NAVY = colors.HexColor("#1f3a5f")
ss = getSampleStyleSheet()
BODY = ParagraphStyle("b", parent=ss["BodyText"], fontSize=9.6, leading=13.5, spaceAfter=5)
H1 = ParagraphStyle("h1", parent=ss["Heading1"], fontSize=14, textColor=NAVY, spaceBefore=12, spaceAfter=6)
H2 = ParagraphStyle("h2", parent=ss["Heading2"], fontSize=11, textColor=NAVY, spaceBefore=8, spaceAfter=4)
CAP = ParagraphStyle("cap", parent=BODY, fontSize=8, textColor=colors.grey, alignment=TA_CENTER)
TITLE = ParagraphStyle("t", parent=ss["Title"], fontSize=22, textColor=NAVY, leading=26, spaceAfter=4)
SUB = ParagraphStyle("s", parent=BODY, fontSize=11, textColor=colors.HexColor("#485563"), alignment=TA_CENTER)
CELL = ParagraphStyle("c", parent=BODY, fontSize=8, leading=10, spaceAfter=0)
CODE = ParagraphStyle("code", fontName="Courier", fontSize=6.4, leading=7.6,
      backColor=colors.HexColor("#f5f7fa"),
                      borderPadding=4)
J = lambda n: json.loads((RESULTS / n).read_text())
esc = lambda s: str(s).replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
P = lambda t, st=BODY: Paragraph(t, st)


def bullets(items):
    return [Paragraph("&bull; " + i, ParagraphStyle("bl", parent=BODY, leftIndent=14, firstLineIndent=-9,
      spaceAfter=2))
            for i in items]


def tbl(rows, widths=None, header=True, fs=8):
    data = [[Paragraph(esc(c) if not str(c).startswith("<") else c, CELL) for c in r] for r in rows]
    t = Table(data, colWidths=widths, repeatRows=1 if header else 0)
    st = [("GRID", (0, 0), (-1, -1), .4, colors.HexColor("#c8d1db")), ("VALIGN", (0, 0), (-1, -1), "TOP"),
          ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f4f7fa")]),
          ("TOPPADDING", (0, 0), (-1, -1), 3), ("BOTTOMPADDING", (0, 0), (-1, -1), 3)]
    if header:
        st += [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("TEXTCOLOR", (0, 0), (-1, 0), colors.white)]
        for j in range(len(rows[0])):
            data[0][j] = Paragraph(f'<font color="white"><b>{esc(rows[0][j])}</b></font>', CELL)
        t = Table(data, colWidths=widths, repeatRows=1)
    t.setStyle(TableStyle(st)); return t


def fig(name, width=16.5, caption=None, root=FIGS):
    p = Path(root) / name
    w, h = PILImage.open(p).size
    items = [Image(str(p), width=width * cm, height=width * cm * h / w)]
    if caption:
        items.append(P(caption, CAP))
    return KeepTogether(items)


def code(*files):
    out = [PageBreak(), P("Appendix - Source code", H1)]
    for f in files:
        path = ROOT / f
        out.append(P(f"<b>{esc(f)}</b>", H2))
        lines = []
        for ln in path.read_text().splitlines():
            lines += textwrap.wrap(ln.replace("\t", "    "), 128, subsequent_indent="        ",
                                   drop_whitespace=False) or [""]
        chunk = 70                      # split into chunks so pages break cleanly
        for i in range(0, len(lines), chunk):
            out.append(Preformatted("\n".join(lines[i:i + chunk]), CODE))
        out.append(Spacer(1, 6))
    return out


def build(fname, num, title, objective, body, code_files):
    def deco(c, d):
        c.saveState(); c.setFont("Helvetica", 7.5); c.setFillColor(colors.grey)
        c.drawString(2 * cm, 1.1 * cm, f"Predictive Analytics & AI Model Deployment  |  Task {num}: {title}")
        c.drawRightString(A4[0] - 2 * cm, 1.1 * cm, f"Page {d.page}"); c.restoreState()
    doc = SimpleDocTemplate(str(OUT / fname), pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
                            topMargin=1.8 * cm, bottomMargin=1.8 * cm, title=f"Task {num} - {title}",
                            author="AI Pipeline Project")
    head = [Spacer(1, 10), P(f"Task {num}", SUB), P(esc(title), TITLE),
            P("Predictive Analytics &amp; AI Model Deployment - Customer Churn Prediction", SUB), Spacer(1, 10),
            Table([[P("<b>Objective.</b> " + objective)]], colWidths=[17 * cm],
                  style=[("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#eaf1f8")), ("BOX", (0, 0), (-1, -1),
```

```
        .6, NAVY),
                        ("LEFTPADDING", (0, 0), (-1, -1), 8), ("TOPPADDING", (0, 0), (-1, -1), 6)])]
    doc.build(head + body + (code(*code_files) if code_files else []), onFirstPage=deco, onLaterPages=deco)
    print("built", fname)


def main():
    t1, t2, t3, t4, t5, dl = (J("task1_collection.json"), J("task2_preprocess.json"), J("task3_training.json"),
                              J("task4_comparison.json"), J("task5_analytics.json"),
      J("task5_deep_learning.json"))
    meta = json.loads((MODELS / "metadata.json").read_text())
    bm = meta["test_metrics_at_threshold"]

    # ---------------- Task 1 ----------------
    desc = {"customer_id": "Unique customer key", "age": "Customer age (years)", "gender": "Male / Female /
      Other",
            "region": "North, South, East, West, Central", "tenure_months": "Months as a customer",
            "contract_type": "Monthly / One-Year / Two-Year", "internet_service": "DSL / Fiber / No Service",
            "payment_method": "Electronic Check, Credit Card, Bank Transfer, Mailed Check",
            "num_products": "Number of subscribed products (1-6)", "monthly_charges": "Current monthly bill",
            "total_charges": "Lifetime billed amount", "num_support_tickets": "Support tickets raised",
            "late_payments": "Count of late payments", "avg_monthly_usage_gb": "Average monthly data usage
      (GB)",
            "satisfaction_score": "Survey score 1-10", "last_login_days": "Days since last login",
            "discount_offer": "Received discount (0/1)", "churn": "TARGET - 1 = customer left"}
    schema = [["Column", "Type", "Description"]] + [[c, t1["dtypes"][c], desc[c]] for c in t1["head_cols"]]
    sample = [t1["head_cols"][:9]] + [r[:9] for r in t1["head"][:5]]
    body = [P("1. Data source and approach", H1),
            P("The task asks for a large dataset from business, healthcare, finance or sales. The chosen domain
      is a "
              "<b>subscription business (sales + finance)</b>: predict customer churn from billing, contract,
      payment, "
              "support and engagement data. The build environment had <b>no internet access</b>, so the data
      could not be "
              "downloaded from a public repository. Instead a reproducible generator (<font
      face='Courier'>src/data_collection.py</font>, "
              "fixed random seed) creates a realistic dataset. <b>The data is synthetic</b>; a real CSV with the
       same columns can be "
              "dropped into <font face='Courier'>data/raw/</font> without changing any other code."),
            P("How the data is generated", H2)] + bullets([
        "Demographic, contract, service and payment attributes are sampled from realistic categorical
      distributions.",
        "Monthly charges depend on internet service type and number of products; total charges = monthly x
      tenure (+ noise).",
        "Support tickets, late payments and satisfaction are inter-correlated (more tickets/late payments ->
      lower satisfaction).",
        "Churn is drawn from a probability that increases with month-to-month contracts, low tenure, high bills,
       tickets, "
        "late payments, inactivity and low satisfaction, and decreases with long contracts, more products and
      discounts. "
        "The intercept is calibrated by bisection so overall churn is about 27%."]) + [
        P("Deliberate data-quality problems (to be fixed in Task 2)", H2)] + bullets([
        "Missing values in 5 columns (%s cells in total)." % f"{t1['missing_cells']:,}",
        "About 0.6% of monthly charges inflated 5-10x (outliers) and 0.3% of ages set to an impossible value
      (250).",
        "Inconsistent text: 'MALE' upper-case genders and ' monthly ' contract labels with stray spaces.",
        "%d exact duplicate rows appended." % t1["duplicate_rows"]]) + [
        P("2. Dataset summary", H1),
        tbl([["Rows", "Columns", "File size", "Churn rate", "Missing cells", "Duplicates"],
             [f"{t1['rows']:,}", t1["columns"], f"{t1['size_mb']} MB", f"{t1['churn_rate_raw']*100:.1f}%",
              f"{t1['missing_cells']:,}", t1["duplicate_rows"]]]),
        Spacer(1, 6), P("Missing values per column: " + ", ".join(f"{k}: {v}" for k, v in
      t1["missing_by_column"].items()) + "."),
        P("3. Data dictionary", H1), tbl(schema, [4 * cm, 2.2 * cm, 10.8 * cm]),
        P("4. Sample rows (first 9 columns)", H1), tbl(sample), Spacer(1, 6),
        P("5. Output", H1)] + bullets(["<font face='Courier'>data/raw/customer_churn_raw.csv</font> - the raw
      dataset.",
                                       "<font face='Courier'>results/task1_collection.json</font> - collection
      summary."])
    build("Task_1_Data_Collection.pdf", 1, "Data Collection", "Collect a large dataset related to business,
      healthcare, finance or sales.",
          body, ["src/data_collection.py"])

    # ---------------- Task 2 ----------------
    a = t2["audit_before"]
    audit = [["Issue", "Found in raw data", "Fix applied"],
             ["Duplicate rows", a["duplicates"], "drop_duplicates before splitting"],
             ["Missing values", ", ".join(f"{k} ({v})" for k, v in a["missing"].items()), "Median (numeric) /
      mode (categorical) imputation inside the pipeline"],
             ["Out-of-range values", ", ".join(f"{k} ({v})" for k, v in a["out_of_range"].items()), "Validated
      against plausible ranges, set to NaN, then imputed"],
             ["Inconsistent gender text", a["dirty_gender_values"], "strip + Title Case normalisation"],
             ["Inconsistent contract text", a["dirty_contract_values"], "strip + Title Case normalisation"]]
    feats = [["Engineered feature", "Formula", "Business meaning"],
             ["charge_per_product", "monthly_charges / num_products", "Price paid per product"],
             ["billing_consistency", "total_charges / (monthly x tenure + 1)", "Detects billing anomalies / plan
       changes"],
             ["ticket_rate", "tickets / (tenure + 1) x 12", "Support tickets per year of relationship"],
             ["late_rate", "late_payments / (tenure + 1) x 12", "Payment-risk intensity"],
             ["engagement_index", "usage_gb / (days_since_login + 1)", "Recent activity vs. volume"],
             ["low_satisfaction", "satisfaction <= 4", "Flag for unhappy customers"],
             ["monthly_fiber_flag", "Monthly contract AND Fiber", "Expensive, uncommitted segment"],
```

```
158             ["tenure_group", "0-12, 13-24, 25-48, 49+", "Non-linear tenure effect"],
159             ["age_group", "<=30, 31-45, 46-60, 60+", "Age band"]]
160     body = [P("1. Pipeline overview", H1)] + bullets([
161         "<b>Audit</b> the raw data (duplicates, missing values, impossible values, inconsistent labels).",
162         "<b>Clean</b>: remove %d duplicates -> %s clean rows." % (a["duplicates"], f"{t2['clean_rows']:,}"),
163         "<b>Split</b> 80/20, stratified on churn: %s train / %s test rows (churn %.1f%% in both)." % (
164             f"{t2['train_rows']:,}", f"{t2['test_rows']:,}", t2["churn_rate_train"] * 100),
165         "<b>Feature engineering</b>: 9 new features (table below) on top of 16 raw ones.",
166         "<b>Encode &amp;amp; scale</b>: median imputation + standardisation for numerics; mode imputation + one-hot
      for categoricals.",
167         "<b>No leakage</b>: all statistics (medians, means, categories) are learned from the training set only.
      The final feature "
168         "matrix is %d rows x %d columns with %d missing values." % (t2["final_matrix_shape"][0],
      t2["final_matrix_shape"][1], t2["final_missing"])]) + [
169         P("2. Data-quality audit", H1), tbl(audit, [3.6 * cm, 5.6 * cm, 7.8 * cm]),
170         fig("t2_missing_values.png", 15.5, "Figure 1 - Missing values before and after range validation"),
171         fig("t2_outliers.png", 13.5, "Figure 2 - Outliers in monthly_charges removed by range validation"),
172         P("3. Feature engineering", H1), tbl(feats, [3.6 * cm, 6 * cm, 7.4 * cm]),
173         P("4. Exploratory analysis", H1),
174         P("Churn is strongly linked to contract type, tenure and payment method (Figure 3). The engineered "
175           "<i>monthly_fiber_flag</i> has the largest correlation with churn of any numeric feature (r = %.2f),
      followed by "
176           "satisfaction score (%.2f) and tenure (%.2f) - evidence that the engineered features add signal
      (Figure 4)." % (
177               t2["top_corr"]["monthly_fiber_flag"], t2["top_corr"]["satisfaction_score"],
      t2["top_corr"]["tenure_months"])),
178         fig("t2_churn_by_segment.png", 17, "Figure 3 - Churn rate by contract, tenure group and payment
      method"),
179         fig("t2_target_corr.png", 11, "Figure 4 - Correlation with churn (* = engineered feature)"),
180         fig("t2_correlation.png", 13, "Figure 5 - Correlation matrix of numeric features"),
181         P("5. Output", H1)] + bullets(["<font face='Courier'>data/processed/train.csv</font> and <font
      face='Courier'>test.csv</font>.",
182                                        "<font face='Courier'>src/features.py</font> - the reusable cleaning +
      feature pipeline used by training and the API."])
183     build("Task_2_Preprocessing_Feature_Engineering.pdf", 2, "Advanced Data Preprocessing and Feature
      Engineering",
184           "Clean the data, handle missing values and outliers, encode/scale, and engineer informative
      features.", body,
185           ["src/features.py", "src/preprocess.py"])
186
187     # ---------------- Task 3 ----------------
188     hp = {"logistic_regression": ("Logistic Regression", "max_iter=2000, C=1.0"),
189           "decision_tree": ("Decision Tree", "max_depth=7, min_samples_leaf=40"),
190           "random_forest": ("Random Forest", "200 trees, min_samples_leaf=5, max_features=sqrt"),
191           "gradient_boosting": ("Gradient Boosting (HistGB)", "lr=0.06, 250 iterations"),
192           "knn": ("k-Nearest Neighbours", "k=35, distance-weighted"),
193           "gradient_boosting_tuned": ("Gradient Boosting - tuned", "RandomizedSearchCV, 10 candidates x 3
      folds")}
194     rows = [["Model", "Key hyper-parameters", "CV ROC-AUC (mean)", "+/- std", "Time (s)"]]
195     for k, (n, h) in hp.items():
196         r = t3[k]; rows.append([n, h, f"{r['cv_roc_auc_mean']:.4f}", "" if r["cv_roc_auc_std"] is None else
      f"{r['cv_roc_auc_std']:.4f}", r["train_seconds"]])
197     bp = t3["gradient_boosting_tuned"]["best_params"]
198     body = [P("1. Approach", H1)] + bullets([
199         "Six candidate models spanning linear, tree, ensemble, boosting and distance-based families, all with
      scikit-learn.",
200         "Each model is wrapped in a <b>single Pipeline</b> (feature engineering -> preprocessing -> estimator)
      so cross-validation "
201         "re-fits the preprocessing on every fold: no data leakage.",
202         "<b>Stratified 3-fold cross-validation</b> on the training set, scored by ROC-AUC (threshold-independent
       and robust to the 27% class imbalance).",
203         "Every fitted pipeline is persisted with joblib to <font face='Courier'>models/</font>."]) + [
204         P("2. Cross-validation results (training set)", H1), tbl(rows, [3.7 * cm, 6.6 * cm, 2.6 * cm, 2 * cm,
      2.1 * cm]),
205         Spacer(1, 6),
206         P("Best tuned parameters for gradient boosting: " + ", ".join(f"{k}={v}" for k, v in bp.items()) + "."),
207         P("3. Observations", H1)] + bullets([
208         "Logistic regression is the strongest single model in CV (%.3f) - the churn signal in this dataset is
      largely additive, which "
209         "a linear model captures well. Regularised, tuned boosting is close behind (%.3f)." % (
210             t3["logistic_regression"]["cv_roc_auc_mean"], t3["gradient_boosting_tuned"]["cv_roc_auc_mean"]),
211         "The un-tuned decision tree and k-NN are weakest (%.3f, %.3f): a single tree overfits and k-NN suffers
      in 44 encoded dimensions." % (
212             t3["decision_tree"]["cv_roc_auc_mean"], t3["knn"]["cv_roc_auc_mean"]),
213         "Tuning helped boosting (%.3f -> %.3f) mainly by using shallower trees (8 leaves) and larger leaf sizes,
       i.e. more regularisation." % (
214             t3["gradient_boosting"]["cv_roc_auc_mean"], t3["gradient_boosting_tuned"]["cv_roc_auc_mean"]),
215         "Standard deviations across folds are small (about 0.001-0.005), so the ranking differences of >0.01 are
       meaningful."]) + [
216         P("4. Output", H1)] + bullets(["<font face='Courier'>models/*.joblib</font> - trained pipelines.",
      "<font face='Courier'>results/task3_training.json</font>."])
217     build("Task_3_Train_Multiple_ML_Models.pdf", 3, "Train Multiple Machine Learning Models (scikit-learn)",
218           "Train several ML models with scikit-learn and tune the most promising one.", body,
      ["src/train_models.py"])
219
220     # ---------------- Task 4 ----------------
221     cols = ["name", "accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc", "log_loss", "brier"]
222     rows = [["Model", "Accuracy", "Precision", "Recall", "F1", "ROC-AUC", "PR-AUC", "Log-loss", "Brier"]] + [
223         [r[c] if c == "name" else f"{r[c]:.3f}" for c in cols] for r in t4["table"]]
224     conf = t4["confusion_matrix"]
225     body = [P("1. Method", H1), P(
226         "All %d saved models were scored on the <b>same held-out test set</b> (%s customers, %.1f%% churn) that
```

```
        was never used for "
227       "fitting or tuning. Eight metrics were computed. Because only %.1f%% of customers stay if we predict 'no
         churn' for everybody, "
228       "accuracy alone is misleading (baseline = %.3f); ROC-AUC, PR-AUC and recall on the churn class matter
        more." % (
229             len(t4["table"]), f"{t4['test_rows']:,}", t4["test_churn_rate"] * 100, (1 - t4["test_churn_rate"]) *
         100,
230           t4["majority_baseline_accuracy"])),
231       P("2. Results (threshold 0.5, sorted by ROC-AUC)", H1), tbl(rows, [3.3 * cm, 1.9 * cm, 1.8 * cm, 1.5 *
        cm, 1.2 * cm, 1.9 * cm, 1.7 * cm, 1.9 * cm, 1.5 * cm]),
232       Spacer(1, 6), fig("t4_metric_bars.png", 16, "Figure 1 - Metrics for all models"),
233       fig("t4_roc.png", 11.5, "Figure 2 - ROC curves"), fig("t4_pr.png", 11.5, "Figure 3 - Precision-Recall
        curves"),
234       P("3. Model selection and decision threshold", H1),
235       P("<b>Selected model: %s</b> (highest ROC-AUC %.3f, lowest log-loss and Brier score, i.e. also the best-
        calibrated probabilities). "
236           "At the default 0.5 threshold recall for churners is low (%.0f%%) because churners are a minority. The
         threshold was therefore tuned "
237           "to maximise F1 on <i>out-of-fold training predictions</i> (not on the test set): <b>threshold =
        %.2f</b>." % (
238             meta["best_model_name"], bm["roc_auc"], t4["table"][0]["recall"] * 100, meta["threshold"])),
239       tbl([["Metric at threshold %.2f" % meta["threshold"], "Accuracy", "Precision", "Recall", "F1", "ROC-
        AUC"],
240            ["Test set", f"{bm['accuracy']:.3f}", f"{bm['precision']:.3f}", f"{bm['recall']:.3f}",
        f"{bm['f1']:.3f}", f"{bm['roc_auc']:.3f}"]]),
241       Spacer(1, 6),
242       P("Confusion matrix on the test set: %d true negatives, %d false positives, %d false negatives, %d true
        positives. "
243           "The lower threshold trades some precision for far more churners caught (recall %.0f%%), which is the
        right trade when a retention "
244           "offer is cheap compared with losing a customer." % (conf[0][0], conf[0][1], conf[1][0], conf[1][1],
        bm["recall"] * 100)),
245       fig("t4_confusion_calibration.png", 16.5, "Figure 4 - Confusion matrix (selected model) and calibration
        curves"),
246       P("4. Interpretation and caveats", H1)] + bullets([
247       "Differences between the top models are small (ROC-AUC 0.78-0.79). Simple logistic regression wins
        because the synthetic churn "
248       "mechanism is close to additive; on real, messier data non-linear models (boosting) often win - the
        pipeline makes it a one-line change.",
249       "Test-set size (6,000) gives an AUC standard error of roughly 0.007, so the top three models are
        statistically close; "
250       "logistic regression is preferred also for interpretability and speed.",
251       "A ceiling near 0.79 AUC is expected: the generator adds random noise to the churn probability, so some
        churn is inherently unpredictable."])
252   build("Task_4_Compare_Model_Performance.pdf", 4, "Compare Model Accuracy and Performance Metrics",
253         "Compare all trained models with consistent metrics on unseen data and select the best one.", body,
        ["src/compare_models.py"])
254
255   # ---------------- Task 5 ----------------
256   imp = [["Rank", "Feature", "AUC drop when shuffled"]] + [[i + 1, r["feature"], f"{r['importance']:.4f} (+/-
        {r['std']:.4f})"] for i, r in enumerate(t5["importance"])]
257   dec = [["Decile", "Customers", "Churn rate", "Lift", "Cum. % of churners captured"]] + [
258       [int(d["decile"]), int(d["customers"]), f"{d['churn_rate']*100:.1f}%", f"{d['lift']:.2f}",
        f"{d['cum_gain']*100:.1f}%"] for d in t5["deciles"]]
259   seg = [["Risk segment", "Customers", "Share", "Avg predicted", "Actual churn rate"]] + [
260       [s["risk_segment"], int(s["customers"]), f"{s['share']*100:.1f}%", f"{s['avg_pred']*100:.1f}%",
        f"{s['actual_rate']*100:.1f}%"] for s in t5["segments"]]
261   roi = [["Top % contacted", "Customers", "Churners reached", "Expected saved", "Net value ($)"]] + [
262       [r["top_pct"], r["contacted"], r["churners_reached"], r["expected_saved"], f"{r['net_value']:,.0f}"] for
         r in t5["roi"]]
263   av = [["Architecture (hidden layers)", "Validation ROC-AUC"]] + [[k, v] for k, v in
        dl["architectures_val_auc"].items()]
264   body = [P("Part A - Deep learning (neural network)", H1),
265           P("TensorFlow and PyTorch could not be installed offline, so the deep-learning component uses
        scikit-learn's "
266             "<b>MLPClassifier</b> - a fully-connected feed-forward neural network trained with back-
        propagation. It runs inside the same "
267             "preprocessing pipeline as every other model."), P("Network design", H2)] + bullets([
268       "Input: 44 preprocessed features; hidden layers ReLU; output: sigmoid probability of churn.",
269       "Optimiser Adam (lr 0.001), mini-batches of 256, L2 penalty 0.001, log-loss.",
270       "<b>Early stopping</b> on a 10%% validation split (patience 10) to prevent overfitting: stopped after %d
         epochs." % dl["epochs_run"],
271       "Three architectures were compared on a hold-out split; <b>%s</b> was selected and re-fitted on the full
         training set (%s trainable parameters)." % (
272           "-".join(map(str, dl["layers"])), f"{dl['n_parameters']:,}")]) + [
273       Spacer(1, 4), tbl(av, [8 * cm, 4 * cm]), Spacer(1, 4),
274       fig("t5_mlp_training.png", 15.5, "Figure 1 - Training loss and validation accuracy per epoch"),
275       P("On the test set the MLP reaches ROC-AUC %.3f - on par with boosting but not better than logistic
        regression (Task 4). With ~24,000 "
276           "tabular rows and a mostly additive signal, deep networks have little to gain; they would matter more
        with far larger data, text or image inputs." % (
277             [r for r in t4["table"] if r["model"] == "mlp_neural_net"][0]["roc_auc"])),
278       P("Part B - Predictive analytics on the selected model (%s)" % esc(meta["best_model_name"]), H1),
279       P("1. What drives churn? (permutation importance)", H2),
280       P("Each raw input column is shuffled in turn on a 3,000-row test sample and the resulting drop in ROC-
        AUC is measured. "
281           "Tenure, total charges (which encodes tenure x price) and contract type dominate, followed by
        satisfaction and payment behaviour."),
282       tbl(imp, [1.5 * cm, 5 * cm, 6 * cm]), Spacer(1, 4), fig("t5_importance.png", 11, "Figure 2 - Permutation
         importance"),
283       P("2. How well does the model rank customers? (lift and gains)", H2),
284       P("Sorting test customers by predicted risk, the top decile churns %.2fx more than average, and
```

```
        contacting the top 20%% of customers "
285             "reaches %.1f%% of all churners." % (t5["top_decile_lift"], t5["top_20pct_gain"] * 100)),
286           tbl(dec, [2 * cm, 3 * cm, 3 * cm, 2.5 * cm, 5 * cm]), Spacer(1, 4), fig("t5_lift_gains.png", 15.5,
        "Figure 3 - Lift by decile and cumulative gains"),
287           P("3. Risk segmentation", H2),
288           P("Predicted probabilities are grouped into Low (&lt;25%), Medium (25-55%) and High (&ge;55%). Predicted
         and actual churn rates agree closely in each "
289             "segment, confirming the probabilities are well calibrated and can be used directly for targeting."),
290           tbl(seg, [3.5 * cm, 3 * cm, 2.5 * cm, 3.5 * cm, 3.5 * cm]), Spacer(1, 4), fig("t5_segments.png", 10.5,
        "Figure 4 - Predicted vs. actual churn by segment"),
291           P("4. Business impact simulation (illustrative)", H2),
292           P("Assumptions (placeholders, replace with real numbers): $%.0f cost per retention offer, $%.0f value
        per retained customer, %.0f%% of contacted churners retained." % (
293               t5["assumptions"]["cost_per_offer"], t5["assumptions"]["value_of_saved_customer"],
        t5["assumptions"]["save_rate"] * 100)),
294           tbl(roi, [3.3 * cm, 3 * cm, 3.6 * cm, 3.3 * cm, 3.3 * cm]), Spacer(1, 4),
295           P("Under these assumptions targeting by model score yields positive net value at every contact level;
        the marginal gain flattens beyond ~40% "
296             "of customers, so the optimal campaign size depends on offer cost and capacity.")]
297       build("Task_5_Deep_Learning_Predictive_Analytics.pdf", 5, "Deep Learning and Predictive Analytics
        Techniques",
298             "Implement a neural network and predictive-analytics techniques (importance, lift, segmentation,
        ROI).", body,
299             ["src/deep_learning.py", "src/predictive_analytics.py"])
300
301       # ---------------- Task 6 ----------------
302       live = (RESULTS / "task6_live_predict.json").read_text().strip(); err = (RESULTS /
          "task6_live_error.json").read_text().strip()
303       log = (RESULTS / "task6_api_test_log.txt").read_text().splitlines()
304       log = [l for l in log if "ResourceWarning" not in l and "assertEqual" not in l and "tracemalloc" not in l]
305       ep = [["Method", "Endpoint", "Description"],
306             ["GET", "/", "Web form: enter customer details, see churn probability and risk level"],
307             ["GET", "/health", "Liveness probe: {status: ok, model: ...}"],
308             ["GET", "/model-info", "Model name, threshold, test metrics, required/optional fields"],
309             ["POST", "/predict", "One customer (JSON) -> probability, label, risk level"],
310             ["POST", "/predict_batch", "JSON list or multipart CSV upload (field 'file'), max 5,000 rows"],
311             ["GET", "/dashboard", "Analytics dashboard (Task 7)"]]
312       body = [P("1. Deployment design", H1)] + bullets([
313           "<b>Flask</b> application factory (<font face='Courier'>app.app:create_app</font>) loads the saved best
          pipeline <b>once</b> at start-up and fails fast if artifacts are missing.",
314           "The saved pipeline contains all preprocessing, so the API accepts <b>raw customer fields</b> (missing
          values, messy text and out-of-range numbers are handled exactly as in training).",
315           "Input validation: three required fields (tenure_months, monthly_charges, contract_type), numeric
          coercion, 5 MB body limit, 5,000-row batch limit; errors return JSON with HTTP 400/413.",
316           "Response: churn probability, decision (threshold %.2f), and risk level Low/Medium/High." %
          meta["threshold"]]) + [
317           P("2. Endpoints", H1), tbl(ep, [2 * cm, 3.6 * cm, 11.4 * cm]),
318           P("3. Live example (real HTTP call against the running server)", H1),
319           P("<b>Request</b>", H2),
320           Preformatted("curl -X POST http://localhost:5000/predict -H 'Content-Type: application/json' \\\n  -d
          '{\"tenure_months\":3,\"monthly_charges\":95,\"total_charges\":285,\n
          \"contract_type\":\"Monthly\",\"internet_service\":\"Fiber\",\n      \"payment_method\":\"Electronic
          Check\",\"satisfaction_score\":2,\n      \"num_support_tickets\":5,\"late_payments\":3}'", CODE),
321           P("<b>Response</b>", H2), Preformatted(live, CODE),
322           P("<b>Validation error (missing fields)</b>", H2), Preformatted(err, CODE),
323           P("4. Automated tests", H1),
324           P("Seven tests (health, pages, high- vs low-risk ordering, messy input, validation errors, JSON batch,
          CSV batch) using Flask's test client. Result:"),
325           Preformatted("\n".join(log[-16:]), CODE),
326           P("5. How to run and deploy", H1),
327           tbl([["Target", "Command / file"],
328                ["Local development", "python -m app.app  ->  http://127.0.0.1:5000"],
329                ["Production (WSGI)", 'gunicorn -b 0.0.0.0:8000 -w 2 "app.app:create_app()"'],
330                ["Docker", "Dockerfile: docker build -t churn-api . && docker run -p 8000:8000 churn-api"],
331                ["Cloud PaaS (Render, Heroku, Railway, ...)", "Procfile + requirements.txt are included; connect
          the GitHub repo and deploy"],
332                ["Cloud containers (AWS ECS/App Runner, GCP Cloud Run, Azure)", "Push the Docker image and expose
          port 8000"]], [6 * cm, 11 * cm]),
333           Spacer(1, 6), P("Note: the API was tested locally and with the test client here; deployment to a public
          cloud requires the reader's own cloud account. "
334                           "Add authentication, HTTPS and monitoring before exposing it publicly.")]
335       build("Task_6_Model_Deployment_Flask_API.pdf", 6, "Deploy the Trained Model (Flask REST API)",
336             "Serve the trained model through a REST API and web interface, ready for Docker / cloud deployment.",
          body,
337             ["app/app.py", "src/predict.py", "tests/test_api.py", "Dockerfile"])
338
339       # ---------------- Task 7 ----------------
340       body = [P("1. What was built", H1), P(
341           "A single self-contained HTML dashboard (<font face='Courier'>reports/dashboard.html</font>, 0.9 MB,
          charts embedded as images, no internet needed) "
342           "plus the <font face='Courier'>reports/test_predictions.csv</font> file behind it. It is generated by
          <font face='Courier'>src/build_dashboard.py</font> and "
343           "served by the API at <font face='Courier'>/dashboard</font>."), P("Dashboard sections", H2)] +
          bullets([
344           "<b>KPI cards</b>: customers analysed, overall churn rate, best model, test ROC-AUC, recall at the
          chosen threshold, number of high-risk customers.",
345           "<b>Data and drivers</b>: churn by contract / tenure / payment; correlation with churn.",
346           "<b>Model comparison</b>: metrics table, grouped bar chart and ROC curves for all models.",
347           "<b>Prediction analysis</b>: confusion matrix + calibration, permutation importance, lift and gains,
          predicted-vs-actual by risk segment.",
348           "<b>Risk segments</b> table and an illustrative <b>retention-campaign ROI</b> table.",
349           "<b>Top-15 highest-risk customers</b> in the test set (a ready-made call list).",
```

```
          "<b>Neural-network training curves</b>."]) + [
          P("2. Key charts reproduced from the dashboard", H1),
          fig("t4_metric_bars.png", 15, "Model comparison chart"), fig("t5_lift_gains.png", 15.5, "Lift and
        cumulative gains"),
          fig("t5_importance.png", 10.5, "Feature importance"), fig("t5_segments.png", 10, "Risk segments"),
          P("3. Reading the dashboard - prediction analysis summary", H1)] + bullets([
          "The selected %s model ranks customers well: ROC-AUC %.3f, top-decile lift %.2fx, and the top 20%% of
        customers hold %.1f%% of churners." % (
              esc(meta["best_model_name"]), bm["roc_auc"], t5["top_decile_lift"], t5["top_20pct_gain"] * 100),
          "Tenure, contract type and total charges are the main drivers; satisfaction score and payment behaviour
        add smaller effects.",
          "Predicted and actual churn rates match in each risk segment, so the segment labels can be trusted
        operationally.",
          "Recommended use: contact High-risk customers first; review Medium risk when capacity allows."]) + [
          P("4. Output files", H1)] + bullets(["<font face='Courier'>reports/dashboard.html</font> - open in any
        browser.",
                                              "<font face='Courier'>reports/figures/*.png</font> - individual
        charts.",
                                              "<font face='Courier'>reports/test_predictions.csv</font> -
        probability, decision and segment per test customer."])
      build("Task_7_Dashboards_and_Reports.pdf", 7, "Dashboards and Reports for Prediction Analysis",
            "Create dashboards and reports that explain model predictions to business users.", body,
          ["src/build_dashboard.py"])

      # ---------------- Task 8 ----------------
      tree = """churn_ml_project/
|-- run_pipeline.py            run Tasks 1-5, 7, 8 end-to-end
|-- requirements.txt  Dockerfile  Procfile  README.md
|-- src/
|   |-- config.py              paths, constants, risk bands
|   |-- features.py            cleaning + feature engineering + sklearn pipeline
|   |-- data_collection.py     Task 1        preprocess.py        Task 2
|   |-- train_models.py        Task 3        compare_models.py    Task 4
|   |-- deep_learning.py       Task 5a       predictive_analytics.py  Task 5b
|   |-- predict.py             inference     build_dashboard.py   Task 7
|   `-- build_docs.py / build_pdfs.py        documentation builders
|-- app/  app.py, templates/index.html       Task 6 (Flask API + UI)
|-- tests/test_api.py
|-- data/ raw/  processed/     models/ *.joblib, best_model.joblib, metadata.json
|-- results/  JSON/CSV outputs   reports/ figures/, dashboard.html
`-- docs/ architecture.png, ARCHITECTURE.md, pdfs/"""
      stages = [["#", "Stage", "Input", "Output", "Module"],
                ["1", "Data collection", "generator / real CSV", "data/raw csv", "data_collection.py"],
                ["2", "Preprocessing", "raw csv", "train.csv, test.csv, EDA figures", "preprocess.py,
          features.py"],
                ["3", "Model training", "train.csv", "6 fitted pipelines", "train_models.py"],
                ["5a", "Deep learning", "train.csv", "MLP pipeline", "deep_learning.py"],
                ["4", "Model comparison", "test.csv + models", "best_model.joblib, metadata.json",
          "compare_models.py"],
                ["5b", "Predictive analytics", "best model + test", "importance, lift, segments, ROI",
          "predictive_analytics.py"],
                ["6", "Deployment", "best_model.joblib", "REST API + web UI", "app/app.py"],
                ["7", "Dashboard", "results + figures", "dashboard.html", "build_dashboard.py"]]
      body = [P("1. Architecture", H1), fig("architecture.png", 17, "Figure 1 - AI pipeline architecture",
          root=DOCS),
              P("2. Workflow", H1), tbl(stages, [1 * cm, 3 * cm, 3.6 * cm, 5 * cm, 4.4 * cm]),
              P("3. Design principles", H1)] + bullets([
            "<b>One pipeline object per model</b> (features + preprocessing + estimator). Training, cross-validation
           and serving use identical code, eliminating train/serve skew and leakage.",
            "<b>Held-out test set</b> used only for final comparison; model threshold selected from out-of-fold
          training predictions.",
            "<b>Reproducibility</b>: fixed seeds, one command (<font face='Courier'>python run_pipeline.py</font>)
          rebuilds all data, models, figures and the dashboard.",
            "<b>Separation of concerns</b>: config, features, training, evaluation, serving and reporting are
          independent modules.",
            "<b>Defensive serving</b>: input validation, tolerant preprocessing (unknown categories ignored, invalid
           numbers imputed), JSON errors."]) + [
            P("4. Project structure", H1), Preformatted(tree, CODE),
            P("5. How to reproduce", H1),
            Preformatted("pip install -r requirements.txt\npython run_pipeline.py           # ~2 minutes\npython -m
           unittest tests.test_api -v\npython -m app.app                # http://127.0.0.1:5000", CODE),
            P("6. Final results at a glance", H1),
            tbl([["Item", "Value"], ["Dataset", "%s customers, 18 columns (synthetic), churn %.1f%%" %
          (f"{t2['clean_rows']:,}", t2["churn_rate_train"] * 100)],
                 ["Features", "%d raw + 9 engineered -> %d model inputs after encoding" % (16,
          t2["encoded_feature_count"])],
                 ["Models trained", "%d (5 classical, 1 tuned boosting, 1 neural network)" % len(t4["table"])],
                 ["Selected model", "%s, threshold %.2f" % (meta["best_model_name"], meta["threshold"])],
                 ["Test ROC-AUC / PR-AUC", "%.3f / %.3f" % (bm["roc_auc"], bm["pr_auc"])],
                 ["Test recall / precision / F1", "%.2f / %.2f / %.2f" % (bm["recall"], bm["precision"], bm["f1"])],
                 ["Top-decile lift", "%.2fx" % t5["top_decile_lift"]]], [5 * cm, 12 * cm]),
            P("7. Limitations and next steps", H1)] + bullets([
            "<b>Synthetic data</b>: results demonstrate a working pipeline, not real-world accuracy. Retrain on real
           data before any business decision.",
            "Deep learning uses scikit-learn's MLP because TensorFlow/PyTorch were unavailable; a Keras model can
          replace the estimator (e.g. via SciKeras).",
            "ROI figures are placeholder assumptions. The API has no authentication, rate limiting or monitoring
          yet.",
            "Next: real data, drift monitoring, MLflow model registry, CI/CD, authentication, A/B test of retention
          offers."]) + [
            P("8. GitHub repository", H1),
            P("A local git repository with an initial commit is included in the project folder. To publish it:")] +
```

```
        [
          Preformatted("cd churn_ml_project\ngit remote add origin https://github.com/<your-username>/<repo-name>.git\ngit branch -M main\ngit push -u origin main", CODE),
          P("The public GitHub URL cannot be created from this environment (no internet / no account access); create an empty repository "
            "on GitHub and run the commands above.")]
    build("Task_8_Pipeline_Architecture_Documentation.pdf", 8, "Document the Complete AI Pipeline Architecture and Workflow",
          "Document the architecture, workflow, design decisions, reproduction steps and limitations of the full AI pipeline.", body,
          ["run_pipeline.py", "src/config.py"])


if __name__ == "__main__":
    main()
```

src/build_source_pdf.py (82 lines)

```
"""Builds ONE PDF containing the complete project source code (all text/code files)."""
import textwrap
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer, Table, TableStyle
from .config import DOCS, ROOT

FILES = ["README.md", "requirements.txt", "Dockerfile", "Procfile", ".gitignore", "run_pipeline.py",
         "src/config.py", "src/features.py", "src/data_collection.py", "src/preprocess.py",
         "src/train_models.py", "src/deep_learning.py", "src/compare_models.py",
         "src/predictive_analytics.py", "src/predict.py", "src/build_dashboard.py", "src/build_docs.py",
         "src/build_pdfs.py", "src/build_source_pdf.py", "app/app.py", "app/templates/index.html",
         "tests/test_api.py", "docs/ARCHITECTURE.md"]
NAVY = colors.HexColor("#1f3a5f")
ss = getSampleStyleSheet()
H1 = ParagraphStyle("h1", parent=ss["Heading1"], fontSize=13, textColor=NAVY, spaceAfter=4)
BODY = ParagraphStyle("b", parent=ss["BodyText"], fontSize=9.5, leading=13)
CODE = ParagraphStyle("c", fontName="Courier", fontSize=6.6, leading=8)
TITLE = ParagraphStyle("t", parent=ss["Title"], fontSize=24, textColor=NAVY, leading=28)


def numbered(path):
    out = []
    for i, ln in enumerate(path.read_text().splitlines(), 1):
        parts = textwrap.wrap(ln.replace("\t", "    "), 112, drop_whitespace=False, subsequent_indent="      ")
          or [""]
        out.append(f"{i:4d}  {parts[0]}")
        out += [f"      {p}" for p in parts[1:]]
    return out


def main():
    dest = DOCS / "pdfs" / "Complete_ML_Project_Source_Code.pdf"

    def deco(c, d):
        c.saveState(); c.setFont("Helvetica", 7.5); c.setFillColor(colors.grey)
        c.drawString(1.6 * cm, 1 * cm, "Complete ML Project Source Code - Customer Churn Prediction &
      Deployment")
        c.drawRightString(A4[0] - 1.6 * cm, 1 * cm, f"Page {d.page}"); c.restoreState()

    doc = SimpleDocTemplate(str(dest), pagesize=A4, leftMargin=1.6 * cm, rightMargin=1.6 * cm,
                            topMargin=1.5 * cm, bottomMargin=1.6 * cm, title="Complete ML Project Source Code")
    counts = {f: len((ROOT / f).read_text().splitlines()) for f in FILES}
    total = sum(counts.values())
    rows = [["#", "File", "Purpose", "Lines"]]
    purpose = {"README.md": "Project overview and quick start", "requirements.txt": "Python dependencies",
               "Dockerfile": "Container image", "Procfile": "PaaS start command", ".gitignore": "Git ignore
      rules",
               "run_pipeline.py": "Runs the whole pipeline", "src/config.py": "Paths and constants",
               "src/features.py": "Cleaning + feature engineering + sklearn pipeline",
               "src/data_collection.py": "Task 1 - dataset", "src/preprocess.py": "Task 2 - preprocessing/EDA",
               "src/train_models.py": "Task 3 - train models", "src/deep_learning.py": "Task 5 - neural
      network",
               "src/compare_models.py": "Task 4 - compare/select model",
               "src/predictive_analytics.py": "Task 5 - importance, lift, segments, ROI",
               "src/predict.py": "Inference helper", "src/build_dashboard.py": "Task 7 - dashboard",
               "src/build_docs.py": "Task 8 - architecture diagram", "src/build_pdfs.py": "Builds the 8 task
      PDFs",
               "src/build_source_pdf.py": "Builds this PDF", "app/app.py": "Task 6 - Flask REST API",
               "app/templates/index.html": "Web form UI", "tests/test_api.py": "API tests",
               "docs/ARCHITECTURE.md": "Architecture notes"}
    for i, f in enumerate(FILES, 1):
        rows.append([str(i), f, purpose[f], str(counts[f])])
    t = Table([[Paragraph(str(c), ParagraphStyle("x", parent=BODY, fontSize=8.5, leading=10,
                 textColor=colors.white if r == 0 else colors.black)) for c in row] for r, row in
      enumerate(rows)],
              colWidths=[1 * cm, 5.6 * cm, 9.2 * cm, 1.6 * cm])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), NAVY), ("GRID", (0, 0), (-1, -1), .4,
      colors.HexColor("#c8d1db")),
                           ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f4f7fa")])]))
    story = [Spacer(1, 40), Paragraph("Complete ML Project Source Code", TITLE),
             Paragraph("Predictive Analytics &amp;amp; AI Model Deployment - Customer Churn", ParagraphStyle("s",
      parent=BODY, fontSize=12, alignment=1)),
             Spacer(1, 16), Paragraph(f"{len(FILES)} files, {total:,} lines. Every source, configuration and
      documentation file in the project, "
                                      "in execution order. Generated data, trained models and figures are not
      source code and are in the project zip.", BODY),
             Spacer(1, 12), Paragraph("Table of contents", H1), t,
             Spacer(1, 10), Paragraph("Run order: <font face='Courier'>python run_pipeline.py</font> then <font
      face='Courier'>python -m app.app</font>.", BODY)]
    for f in FILES:
        story += [PageBreak(), Paragraph(f"{f}  <font size=8 color='grey'>({counts[f]} lines)</font>", H1)]
        lines = numbered(ROOT / f)
        for i in range(0, len(lines), 78):
            story.append(Preformatted("\n".join(lines[i:i + 78]), CODE))
    doc.build(story, onFirstPage=deco, onLaterPages=deco)
    print(dest.name, len(FILES), "files", total, "lines")
```

```
79
80
81  if __name__ == "__main__":
82      main()
```

app/app.py (92 lines)

```
"""TASK 6 - Model deployment: Flask REST API + web form + dashboard host.

Run locally:      python -m app.app                (http://127.0.0.1:5000)
Run production:   gunicorn -b 0.0.0.0:8000 "app.app:create_app()"
"""
import io
import pandas as pd
from flask import Flask, jsonify, render_template, request, send_from_directory
from src.config import REPORTS
from src.predict import load_artifacts, predict_records

REQUIRED = ["tenure_months", "monthly_charges", "contract_type"]
NUMERIC_FIELDS = ["age", "tenure_months", "monthly_charges", "total_charges", "num_products",
                  "num_support_tickets", "late_payments", "avg_monthly_usage_gb",
                  "satisfaction_score", "last_login_days", "discount_offer"]
MAX_BATCH = 5000


def validate(rec, idx=None):
    where = f"record {idx}: " if idx is not None else ""
    if not isinstance(rec, dict):
        return f"{where}each record must be a JSON object"
    missing = [f for f in REQUIRED if rec.get(f) in (None, "")]
    if missing:
        return f"{where}missing required field(s): {', '.join(missing)}"
    for f in NUMERIC_FIELDS:
        if rec.get(f) not in (None, ""):
            try:
                float(rec[f])
            except (TypeError, ValueError):
                return f"{where}field '{f}' must be numeric"
    return None


def create_app():
    app = Flask(__name__)
    app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
    model, meta = load_artifacts()          # fail fast if artifacts are missing

    @app.get("/")
    def index():
        return render_template("index.html", meta=meta)

    @app.get("/health")
    def health():
        return jsonify(status="ok", model=meta["best_model_name"])

    @app.get("/model-info")
    def model_info():
        return jsonify(model=meta["best_model_name"], threshold=meta["threshold"],
                       test_metrics=meta["test_metrics_at_threshold"], trained_at=meta["trained_at"],
                       required_fields=REQUIRED, numeric_fields=NUMERIC_FIELDS,
                       categorical_fields=["gender", "region", "contract_type", "internet_service",
      "payment_method"])

    @app.post("/predict")
    def predict():
        rec = request.get_json(silent=True)
        err = validate(rec)
        if err:
            return jsonify(error=err), 400
        return jsonify(predict_records([rec])[0])

    @app.post("/predict_batch")
    def predict_batch():
        if request.files.get("file"):                       # CSV upload
            try:
                df = pd.read_csv(io.BytesIO(request.files["file"].read()))
            except Exception:
                return jsonify(error="could not parse CSV"), 400
            records = df.where(df.notna(), None).to_dict("records")
        else:
            records = request.get_json(silent=True)
        if not isinstance(records, list) or not records:
            return jsonify(error="send a non-empty JSON list of records or a CSV file"), 400
        if len(records) > MAX_BATCH:
            return jsonify(error=f"batch too large (max {MAX_BATCH})"), 413
        for i, r in enumerate(records):
            err = validate(r, i)
            if err:
                return jsonify(error=err), 400
        results = predict_records(records)
        return jsonify(count=len(results), results=results)

    @app.get("/dashboard")
    def dashboard():
        return send_from_directory(REPORTS, "dashboard.html")

    return app
```

```
89
90
91  if __name__ == "__main__":
92      create_app().run(host="0.0.0.0", port=5000, debug=False)
```

app/templates/index.html (37 lines)

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Churn Prediction</title>
<style>
body{font-family:system-ui,Segoe UI,Arial,sans-serif;background:#f4f6f8;margin:0;color:#1d2733}
header{background:#1f3a5f;color:#fff;padding:18px 24px}header h1{margin:0;font-size:20px}header p{margin:4px 0 0;opacity:.8;font-size:13px}
main{max-width:860px;margin:20px auto;padding:0 16px}
.card{background:#fff;border-radius:10px;padding:20px;box-shadow:0 1px 4px rgba(0,0,0,.08);margin-bottom:16px}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(190px,1fr));gap:12px}
label{font-size:12px;font-weight:600;color:#485563;display:block;margin-bottom:3px}
input,select{width:100%;padding:8px;border:1px solid #cbd3db;border-radius:6px;box-sizing:border-box;font-size:14px}
button{background:#1f3a5f;color:#fff;border:0;border-radius:6px;padding:10px 20px;font-size:15px;cursor:pointer;margin-top:14px}
#result{display:none;text-align:center}#prob{font-size:44px;font-weight:700}
.Low{color:#2e8b57}.Medium{color:#d98e04}.High{color:#c0392b}a{color:#1f3a5f}
</style></head><body>
<header><h1>Customer Churn Prediction</h1>
<p>Model: {{ meta.best_model_name }} &middot; decision threshold {{ meta.threshold }} &middot; test ROC-AUC {{ meta.test_metrics_at_threshold.roc_auc }}</p></header>
<main>
<div class="card"><div class="grid" id="form"></div><button onclick="run()">Predict churn risk</button>
<span style="margin-left:14px;font-size:13px"><a href="/dashboard">Open dashboard</a> &middot; <a href="/model-info">Model info (JSON)</a></span></div>
<div class="card" id="result"><div>Churn probability</div><div id="prob"></div><div id="risk" style="font-size:20px;font-weight:600"></div><div id="label" style="color:#485563;margin-top:6px"></div></div>
</main>
<script>
const F=[["tenure_months","Tenure (months)","number",12],["monthly_charges","Monthly charges","number",85],["total_charges","Total charges","number",1020],
["age","Age","number",35],["num_products","Products","number",2],["num_support_tickets","Support tickets","number",3],["late_payments","Late payments","number",2],
["avg_monthly_usage_gb","Usage (GB/mo)","number",90],["satisfaction_score","Satisfaction (1-10)","number",4],["last_login_days","Days since login","number",25],["discount_offer","Discount (0/1)","number",0],
["contract_type","Contract","select",["Monthly","One-Year","Two-Year"]],["internet_service","Internet","select",["Fiber","DSL","No Service"]],
["payment_method","Payment","select",["Electronic Check","Credit Card","Bank Transfer","Mailed Check"]],["gender","Gender","select",["Female","Male","Other"]],["region","Region","select",["North","South","East","West","Central"]]];
const box=document.getElementById("form");
F.forEach(([k,l,t,d])=>{const w=document.createElement("div");w.innerHTML=`<label>${l}</label>`+(t==="select"?`<select id="${k}">${d.map(o=>`<option>${o}</option>`).join("")}</select>`:`<input id="${k}" type="number" step="any" value="${d}">`);box.appendChild(w)});
async function run(){const rec={};F.forEach(([k])=>{const v=document.getElementById(k).value;rec[k]=v});
const r=await fetch("/predict",{method:"POST",headers:{"Content-Type":"application/json"},body:JSON.stringify(rec)});const j=await r.json();
document.getElementById("result").style.display="block";
if(!r.ok){document.getElementById("prob").textContent="Error";document.getElementById("risk").textContent=j.error;return}
document.getElementById("prob").textContent=(j.churn_probability*100).toFixed(1)+"%";
const e=document.getElementById("risk");e.textContent=j.risk_level+" risk";e.className=j.risk_level;document.getElementById("label").textContent=j.label}
</script></body></html>
```

tests/test_api.py (55 lines)

```
"""API tests (Flask test client). Run: python -m unittest tests.test_api -v"""
import unittest
from app.app import create_app

HIGH_RISK = {"tenure_months": 3, "monthly_charges": 95, "total_charges": 285, "contract_type": "Monthly",
             "internet_service": "Fiber", "payment_method": "Electronic Check", "satisfaction_score": 2,
             "num_support_tickets": 5, "late_payments": 3, "last_login_days": 60, "num_products": 1, "age": 28}
LOW_RISK = {"tenure_months": 60, "monthly_charges": 55, "total_charges": 3300, "contract_type": "Two-Year",
            "internet_service": "DSL", "payment_method": "Credit Card", "satisfaction_score": 9,
            "num_support_tickets": 0, "late_payments": 0, "last_login_days": 2, "num_products": 4,
            "discount_offer": 1, "age": 52}


class ApiTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.c = create_app().test_client()

    def test_health(self):
        r = self.c.get("/health"); self.assertEqual(r.status_code, 200); self.assertEqual(r.json["status"],
      "ok")

    def test_home_and_dashboard(self):
        self.assertEqual(self.c.get("/").status_code, 200)
        self.assertEqual(self.c.get("/dashboard").status_code, 200)

    def test_predict_high_vs_low(self):
        hi = self.c.post("/predict", json=HIGH_RISK).json; lo = self.c.post("/predict", json=LOW_RISK).json
        self.assertGreater(hi["churn_probability"], lo["churn_probability"])
        self.assertEqual(hi["risk_level"], "High"); self.assertEqual(lo["risk_level"], "Low")

    def test_messy_input_is_handled(self):
        r = self.c.post("/predict", json={"tenure_months": "12", "monthly_charges": "70", "contract_type": "
      monthly "})
        self.assertEqual(r.status_code, 200); self.assertTrue(0 <= r.json["churn_probability"] <= 1)

    def test_validation_errors(self):
        self.assertEqual(self.c.post("/predict", json={"tenure_months": 5}).status_code, 400)
        bad = dict(LOW_RISK, monthly_charges="abc")
        self.assertEqual(self.c.post("/predict", json=bad).status_code, 400)
        self.assertEqual(self.c.post("/predict", data="not json").status_code, 400)

    def test_batch(self):
        r = self.c.post("/predict_batch", json=[HIGH_RISK, LOW_RISK])
        self.assertEqual(r.status_code, 200); self.assertEqual(r.json["count"], 2)
        self.assertEqual(self.c.post("/predict_batch", json=[]).status_code, 400)

    def test_batch_csv_upload(self):
        import io
        csv = "tenure_months,monthly_charges,contract_type\n5,90,Monthly\n50,40,Two-Year\n"
        r = self.c.post("/predict_batch", data={"file": (io.BytesIO(csv.encode()), "c.csv")},
                        content_type="multipart/form-data")
        self.assertEqual(r.status_code, 200); self.assertEqual(r.json["count"], 2)


if __name__ == "__main__":
    unittest.main()
```

docs/ARCHITECTURE.md (31 lines)

```
# Architecture & Workflow

![architecture](architecture.png)

## Data flow
1. **Collect** - `src/data_collection.py` -> `data/raw/customer_churn_raw.csv` (30,180 rows incl. 180 duplicates).
2. **Preprocess** - `src/preprocess.py`: audit, de-duplicate, stratified 80/20 split -> `data/processed/{train,test}.csv`.
3. **Feature pipeline** - `src/features.py`: `add_features` (normalise text, null impossible values, engineer 9 features)
   then `ColumnTransformer` (median-impute + scale numerics; mode-impute + one-hot categoricals).
4. **Train** - `src/train_models.py`: 5 classical models + randomized-search-tuned gradient boosting, 3-fold CV.
5. **Deep learning** - `src/deep_learning.py`: MLP architecture selection, early stopping.
6. **Compare** - `src/compare_models.py`: 8 metrics on the held-out test set, best model by ROC-AUC,
   F1-optimal threshold from out-of-fold train predictions -> `models/best_model.joblib`, `metadata.json`.
7. **Analytics** - `src/predictive_analytics.py`: permutation importance, lift/gains, risk segments, ROI simulation.
8. **Serve** - `app/app.py` loads `best_model.joblib` once; `src/predict.py` does inference.
9. **Report** - `src/build_dashboard.py` -> `reports/dashboard.html` (also served at `/dashboard`).

## Key design decisions
- **One pipeline object per model** (features + preprocessing + estimator): identical transformations in training,
  cross-validation and production; preprocessing statistics are learned only from training folds (no leakage).
- **Threshold chosen on out-of-fold train predictions**, never on the test set.
- **Validation at the API edge**: required fields, numeric coercion, batch-size limit, JSON errors with 400/413.
- **Risk segments** (Low < 0.25 <= Medium < 0.55 <= High) are defined once in `src/config.py`.

## Limitations
- Data is synthetic; metrics show the pipeline works, not real-world performance. Re-train on real data before use.
- Deep learning uses scikit-learn's MLP (TensorFlow/PyTorch were not installable offline).
- ROI numbers are illustrative assumptions. No authentication / monitoring is included in the API.

## Next steps
Real data, drift monitoring, model registry (MLflow), auth + rate limiting, CI pipeline, TensorFlow/Keras variant.
```